NTNU

Kunnskap for en bedre verden

Master thesis

Course Code - Course Name

Biologically Plausible Alternatives To Backpropagation For Training Neural Networks

Author:
Mats Wiig Martinussen

Date

Table of Contents

List of Figures

List of Tables

1 Introduction

1.1 Background and Motivation

1.1.1 Recent Progress in Artificial Intelligence

Artificial intelligence has undergone a dramatic transformation over the past decade, emerging as one of the most powerful technological forces of our time. Recent advances in deep learning and large-scale optimization have produced systems capable of performing tasks once considered exclusive to human intelligence. Large language models can generate coherent text, write code, and assist in complex reasoning tasks. Reinforcement learning systems have achieved superhuman performance in games such as chess and Go, while breakthroughs in scientific machine learning have enabled progress on long-standing challenges such as protein structure prediction. At the same time, autonomous systems are transitioning from research prototypes to real-world deployment, with self-driving vehicles now operating in urban environments. Taken together, these developments suggest that artificial intelligence is not only progressing rapidly, but may be approaching a level of capability that raises fundamental questions about the feasibility of artificial general intelligence.

1.1.2 Open Challenges in Current AI Systems

Despite this rapid progress, there is increasing debate within the research community regarding whether current approaches are sufficient to achieve artificial general intelligence. Yann LeCun, often referred to as one of the "godfathers of AI", has been particularly vocal in his criticism of current paradigms, recently co-founding a company backed by over $1 billion to pursue fundamentally different approaches to intelligence. These developments suggest that, despite impressive empirical success, fundamental challenges may still remain. Among others, these include:

- **Energy intensity.** The 10**11 neurons and 1015 synapses contained in the typical human brain expend approximately 20 W of power, whereas a digital simulation of an artificial neural network of approximately the same size consumes 7.9MW. Source: Wong, T. M. et al. 1014. Report no. RJ10502 (ALM1211-004) (IBM, 2012). The power consumption of this simulation of the brain puts that of conventional digital systems into context. You can also include a graph that show the energy consumption per model since 1985. Exponential graph. Cant keep growing like this forever. So maybe right now, the real bottleneck to intelligence is energy. So we need more energy efficient architectures to achieve AGI.
- **Lack of continual learning.** Most modern AI systems lack the ability to learn continuously from experience. During training, models are optimized on a fixed dataset, after which their parameters remain largely static during deployment, preventing them from naturally adapting to new information. While updates through fine-tuning are possible, these often suffer from *catastrophic forgetting*, where newly acquired knowledge interferes with previously learned information. This limitation is problematic because a generally intelligent system should be able to incorporate new information as it becomes available—for example, adapting to a specific user or immediately integrating major real-world events—without degrading prior knowledge. Current approaches such as in-context learning partially address this by conditioning on recent inputs, but this remains a temporary and indirect workaround rather than true persistent learning. As such, the lack of robust continual learning mechanisms represents a key limitation of current AI systems.
- **Limited out-of-distribution generalization.** Current AI systems often perform well on tasks that resemble their training data, but their performance can degrade significantly when faced with novel situations. This is because most models rely on the assumption that training and test data follow similar distributions, an assumption that rarely holds in real-world settings. As a result, models tend to exploit statistical regularities or spurious correlations present in the training data, rather than learning underlying structure that transfers to new

environments. Recent work has shown that even when models appear robust on standard benchmarks, significant failure modes can emerge under distribution shifts, indicating that current evaluation methods may overestimate true generalization performance. Since general intelligence fundamentally requires the ability to apply knowledge to new and unseen situations, limited out-of-distribution generalization represents a central challenge for current AI systems.

- **Limited abstraction.** Current AI systems often struggle to learn high-level, abstract concepts that generalize across different contexts. Instead, they tend to rely on statistical regularities and surface-level features present in the training data, rather than capturing the underlying meaning of a concept. For example, a model trained to recognize chairs may rely on features such as the presence of four legs, and fail to identify unconventional designs that do not match these patterns, despite clearly serving the same purpose. In contrast, the abstract concept of a chair is simply something to sit on, independent of its specific appearance. As a result, models may perform well on familiar inputs but fail to generalize when superficial characteristics change.
- **Lack of grounding in the physical world.** Most modern AI systems learn from static datasets rather than through interaction with the environment, limiting their ability to connect representations to real-world experience. As a result, their knowledge is largely derived from correlations in data rather than direct interaction with the underlying phenomena. For example, a language model may learn about cooking by reading large numbers of recipes, but it has no direct experience of taste, texture, or smell, and therefore lacks a grounded understanding of what the concepts it describes actually correspond to in the real world. This makes their behavior highly dependent on the data they were trained on and limits their ability to reason about concepts that are inherently tied to physical interaction.

1.1.3 Two Perspectives on the Path Toward General Intelligence

There are broadly two perspectives on the trajectory of current AI systems. On the one hand, some researchers argue that existing approaches may be fundamentally limited, lacking key ingredients required for general intelligence. On the other hand, there is a belief that continued scaling of data, model size, and computational resources will lead to increasingly general capabilities, with no clear evidence of saturation so far.

Regardless of which perspective proves correct, systems that achieve general intelligence must exhibit the capabilities outlined above, such as continual learning, strong generalization, and the ability to form abstract and grounded representations. Even if these properties ultimately emerge from scaling alone, it remains unclear whether current approaches represent the most efficient or principled way of achieving them. The natural thing to do is, therefore, to look at systems that already exhibit these capabilities.

1.1.4 Biological Systems as a Reference for General Intelligence

Biological systems provide a proof of existence that the capabilities outlined above can be achieved in practice. Humans and other animals exhibit continual learning, adapting to new experiences without requiring a separate training phase. They demonstrate strong generalization, applying knowledge to novel situations—for example, recognizing objects under new conditions or solving problems they have not explicitly encountered before.

In addition, biological learning is highly sample-efficient. A child can learn the meaning of a new word, such as "butterfly," from only a few exposures by leveraging contextual cues, such as where a speaker is looking or pointing, rather than requiring repeated observations across large datasets. Biological systems also form abstract representations, allowing them to recognize that an object is a chair regardless of its specific appearance, as long as it serves the function of something to sit on.

Finally, biological intelligence is grounded in interaction with the physical world. Concepts such as "heavy" or "taste" are not learned through descriptions alone, but through direct sensory experience and interaction. Together, these properties highlight that the challenges identified in current AI systems are routinely addressed in biological systems, suggesting that they may provide a valuable source of inspiration for developing more general and adaptive learning mechanisms.

1.1.5 Biological Inspiration: Architecture and Learning

Research inspired by biological systems broadly follows two directions. One approach focuses on developing architectures that more closely resemble biological neural systems. This includes neuromorphic hardware **I could show an example or two here, and generally spend some more time on neuromorphic arhcitectures**, but also more radical approaches where biological neurons themselves are used as the computational substrate. A particularly striking example is the CL1 system developed by Cortical Labs, where networks of living neurons are interfaced with a digital environment and trained to perform tasks through real-time interaction. In such systems, neural activity both drives and responds to external inputs, forming a closed feedback loop that allows the system to adapt its behavior over time. Despite consisting of relatively simple neural cultures, these systems have demonstrated the ability to learn goal-directed behavior, highlighting the potential of biological substrates for information processing and learning. While still in an early stage, this line of work represents a fundamentally different paradigm from conventional AI, where computation is no longer confined to silicon, but emerges from the dynamics of living neural systems.

An alternative approach is to focus not on replicating the full biological architecture, but on understanding and emulating the mechanisms by which such systems learn. Rather than building biological systems directly, this direction seeks to identify learning rules that capture key properties of biological learning. In this thesis, we focus on this second direction, and in particular on biologically plausible learning algorithms.

1.1.6 Motivation for Alternative Learning Rules

Backpropagation is not biologically plausible. It relies on mechanisms such as exact gradient computation, symmetric weight transport, and the propagation of global error signals through the network—none of which are known to be implementable in biological neural systems. If architectures that more closely resemble biological systems, such as CL1, turn out to be a viable path toward more general intelligence, then backpropagation is no longer applicable as a learning mechanism. In such settings, alternative learning rules are not just interesting, but necessary.

At the same time, even within current machine learning systems, backpropagation is not without limitations. It is computationally and memory intensive, and its effectiveness relies on highly structured training procedures and large amounts of data. This raises the question of whether alternative learning mechanisms, potentially inspired by biological systems, can offer different trade-offs or point toward new directions for improving existing approaches.

1.2 Contribution, Research Objectives and Research Questions

1.2.1 Contribution

1.2.2 Objectives

The objective of this thesis is to investigate perturbation-based learning methods as biologically plausible alternatives to backpropagation. The thesis combines literature analysis, theoretical development, and empirical evaluation.

- **Establish the current state of perturbation-based learning.** Review and compare

existing perturbation-based learning methods, with emphasis on how different approaches inject noise, form learning signals, and update weights. This provides the foundation for identifying what is already understood, which existing variants are most relevant, and where the literature remains incomplete.

- **Further develop the theory of perturbation-based learning.** Build on the existing literature to clarify the relationship between common perturbation algorithms, and use this understanding to develop a theoretically motivated variant of node perturbation. The aim is to strengthen the theoretical basis of perturbation learning and identify a modification that may improve estimator quality and learning performance.
- **Evaluate perturbation-based methods empirically.** Assess how existing perturbation-based learning methods and the proposed method perform across supervised learning tasks of increasing complexity. The goal is to understand both when these methods work well and when they begin to break down, using backpropagation as the standard reference method.

1.2.3 Research Questions

1.3 Structure of the Thesis

2 Theory

2.1 Theory taken from existing literature

This section addresses the first objective of the thesis by reviewing the relevant perturbation-learning literature and clarifying how existing methods differ in noise placement, learning signal, and update rule.

2.1.1 How the brain learns

Learning in biological systems can be understood as the process of modifying internal models of the world. These internal models allow an organism to interpret sensory inputs, predict future events, and choose actions. In the brain, such models are not stored as explicit symbolic rules, but are encoded in the activity of large networks of neurons and in the strengths of the synaptic connections between them. Learning therefore requires plasticity: the ability of synapses to strengthen or weaken in response to experience (Dehaene, 2020; Kandel et al., 2021).

2.1.1.1 Hebbian learning

At the level of individual synapses, one of the most influential principles is Hebbian learning. Hebb's original idea was that when the activity of a presynaptic neuron repeatedly contributes to the activation of a postsynaptic neuron, the connection between them should become stronger (Hebb, 1949). In mathematical form, this is often expressed as a local update rule

$$\Delta w_{ij} = \eta x_i f(x_j),$$

where w_{ij} is the synaptic weight from presynaptic neuron i to postsynaptic neuron j, x_i is the presynaptic activity, x_j is the postsynaptic activity, $f(\cdot)$ is a function of the postsynaptic state, and η is a learning-rate parameter. The key feature of this rule is locality: the synapse only requires information available at the synapse itself, namely pre- and postsynaptic activity. Stabilized variants of Hebbian learning, such as Oja's rule, show that such local mechanisms can extract meaningful structure from data, for example by learning the first principal component of the input distribution (Oja, 1982).

2.1.1.2 Neuromodulators and three-factor learning

However, purely Hebbian learning is not sufficient as a general model of learning, because it does not include information about whether an outcome was useful, rewarding, or correct. Biological learning often depends on such feedback. A central example is dopamine, which is strongly associated with reward-prediction errors: dopamine activity tends to increase when outcomes are better than expected and decrease when outcomes are worse than expected (Schultz et al., 1997). This type of signal is not specific to a single synapse, but is broadcast more broadly and can modulate plasticity across many synapses.

This motivates three-factor learning rules, where synaptic change depends on three components: presynaptic activity, postsynaptic state, and a global modulatory signal. In this framework, the local pre- and postsynaptic activity creates an eligibility trace at the synapse, which marks the synapse as eligible for modification. A later global signal, such as a reward-prediction error, determines whether this eligible synapse is strengthened or weakened (Frémaux and Gerstner, 2016; Gerstner et al., 2018). A simple abstract form of such a rule is

$$\Delta w_{ij} = \eta e_{ij} M, \qquad e_{ij} = x_i f(x_j),$$

where e_{ij} is the local eligibility trace and M is the global modulatory signal. Three-factor rules are therefore more powerful than purely Hebbian learning because they combine local credit assignment with global information about outcome quality.

2.1.1.3 Stochasticity in biological learning

Biological learning also takes place in a noisy environment. Neural firing is variable, synaptic transmission is stochastic, and the same stimulus does not always produce exactly the same neural response. Rather than being only a nuisance, this stochasticity can provide a source of exploration. If random fluctuations in neural activity or synaptic efficacy are correlated with later reward or error signals, then the nervous system can reinforce fluctuations that improve performance and suppress those that do not (Seung, 2003). This idea directly motivates perturbation-based learning algorithms, where random perturbations are introduced into weights or neural activities and then correlated with a global scalar learning signal.

The biological principles relevant for this thesis can therefore be summarized as follows. Synaptic learning should be local in the sense that each weight update depends on information available at the corresponding synapse. It may also be modulated by a global scalar signal, analogous to a reward, error, or neuromodulatory feedback signal. Finally, stochastic fluctuations in weights or neural activity can provide the exploratory signal needed for learning. These three ingredients—local synaptic information, global feedback, and stochastic perturbations—form the biological motivation for the perturbation-based learning rules studied in the remainder of this chapter.

2.1.2 Learning in artificial neural networks

2.1.2.1 Feedforward neural networks and supervised learning

Artificial neural networks can be viewed as simplified computational models inspired by biological neural systems. Like biological networks, they consist of units connected by weighted connections, and learning takes place by adjusting these weights. In this thesis, we focus on feedforward neural networks, where information passes through the network layer by layer from input to output. For neuron j in layer ℓ, the pre-activation and activation are given by

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell, \qquad x_j^\ell = \phi(z_j^\ell),$$

where W_{ij}^ℓ is the weight from neuron i in layer $\ell - 1$ to neuron j in layer ℓ, b_j^ℓ is a bias term, $\phi(\cdot)$ is a nonlinear activation function, and x_j^ℓ is the resulting neuron activity. By stacking multiple such layers, the network defines a parameterized function $f_\theta(x)$, where θ denotes the collection of all weights and biases (Goodfellow et al., 2016).

In supervised learning, the objective is to adjust the parameters so that the network maps inputs to desired outputs. Given a dataset of input–target pairs (x, y), the network produces a prediction $f_\theta(x)$, and a loss function $L(y, f_\theta(x))$ measures the discrepancy between the prediction and the target. Training then consists of finding parameters that minimize the expected loss,

$$\theta^* = \arg\min_\theta \mathbb{E}_{(x,y)} \left[L(y, f_\theta(x)) \right].$$

This formalizes learning as parameter optimization: the network changes its weights and biases so that its internal mapping better matches the structure of the data. In this sense, artificial neural networks and biological neural systems share a high-level principle: both learn by changing the parameters of a distributed network. The main difference lies in how those parameter changes are computed.

2.1.2.2 Backpropagation

Backpropagation is the standard algorithm for computing gradients in artificial neural networks (Rumelhart et al., 1986). It allows gradient descent to be applied efficiently by using the chain rule to propagate error information backward through the network. For a loss function L, the parameter update is based on the gradient

$$\nabla_\theta L,$$

which specifies how each parameter should change to locally reduce the loss. Once this gradient is computed, standard gradient descent updates the parameters according to

$$\theta \leftarrow \theta - \eta \nabla_\theta L,$$

where $\eta > 0$ is the learning rate.

To derive the backpropagation update, consider a feedforward network with pre-activations z_j^ℓ and activations $x_j^\ell = \phi(z_j^\ell)$. The key quantity in backpropagation is the error term

$$\delta_j^\ell := \frac{\partial L}{\partial z_j^\ell},$$

which measures how a small change in the pre-activation of neuron j in layer ℓ affects the loss. Once δ_j^ℓ is known, the gradient with respect to an incoming weight follows directly from the chain rule:

$$\frac{\partial L}{\partial W_{ij}^\ell} = \frac{\partial L}{\partial z_j^\ell} \frac{\partial z_j^\ell}{\partial W_{ij}^\ell}.$$

Since

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell,$$

we have

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1}.$$

Thus, the weight gradient is

$$\frac{\partial L}{\partial W_{ij}^\ell} = \delta_j^\ell x_i^{\ell-1}.$$

Similarly, the bias gradient is

$$\frac{\partial L}{\partial b_j^\ell} = \delta_j^\ell.$$

These equations show that if the error term δ_j^ℓ is known, the gradients in that layer can be computed using the same local presynaptic activity that appeared in the forward pass.

The remaining question is how to compute the error terms δ_j^ℓ. In the output layer, this term can be computed directly from the loss. For a neuron j in the final layer L, we have

$$\delta_j^L = \frac{\partial L}{\partial z_j^L} = \frac{\partial L}{\partial x_j^L} \phi'(z_j^L).$$

For hidden layers, the loss does not depend on the layer directly, but only through its effect on later layers. Applying the chain rule gives

$$\frac{\partial L}{\partial x_j^\ell} = \sum_k \frac{\partial L}{\partial z_k^{\ell+1}} \frac{\partial z_k^{\ell+1}}{\partial x_j^\ell}.$$

Since

$$z_k^{\ell+1} = \sum_j W_{jk}^{\ell+1} x_j^\ell + b_k^{\ell+1},$$

we obtain

$$\frac{\partial z_k^{\ell+1}}{\partial x_j^\ell} = W_{jk}^{\ell+1}.$$

Therefore,

$$\frac{\partial L}{\partial x_j^\ell} = \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1}.$$

Multiplying by the derivative of the activation function gives the recursive backpropagation equation

$$\delta_j^\ell = \frac{\partial L}{\partial z_j^\ell} = \phi'(z_j^\ell) \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1}.$$

This recursion allows the error terms to be computed layer by layer from the output layer back toward the input layer.

Combining the error recursion with the weight-gradient expression gives the full backpropagation update for a hidden-layer weight:

$$\frac{\partial L}{\partial W_{ij}^\ell} = x_i^{\ell-1} \phi'(z_j^\ell) \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1}.$$

Backpropagation is powerful because it computes the exact gradient of the loss with respect to every parameter using one forward pass and one backward pass. This efficiency is the reason it remains the dominant training algorithm for artificial neural networks. In this thesis, backpropagation is therefore used as the reference method in the empirical evaluation. However, the mechanism by which it computes gradients differs substantially from the local and modulatory learning principles discussed above, motivating the question of whether more biologically plausible alternatives can provide useful learning rules.

2.1.2.3 Why backpropagation is biologically implausible

A literal implementation of backpropagation is biologically implausible because it requires the network to compute exact gradient information through a separate recursive backward calculation. In artificial neural networks, this is straightforward: error terms are computed at the output layer and then propagated backward layer by layer using the chain rule. In biological circuits, however, it is not clear how neurons and synapses would implement such an explicit derivative computation. The brain does not appear to have a separate computational mechanism that stores error variables, multiplies them by downstream weight matrices, and recursively distributes exact gradients to earlier layers. Thus, the central issue is not merely one technical detail of backpropagation, but the broader question of how a biological network could perform the precise layer-wise credit assignment required by the algorithm (Lillicrap et al., 2020).

The difficulty becomes clearer when comparing backpropagation with the biological learning principles discussed above. Three-factor learning rules require local synaptic information, such as presynaptic activity and postsynaptic state, together with a global modulatory signal. Backpropagation instead requires detailed vector-valued error feedback, where each neuron receives a specific error signal describing how its activity should change to reduce the final loss. This makes backpropagation much more informative and efficient than scalar-feedback learning rules, but also much harder to reconcile with biological implementation. A scalar reward or error signal can

plausibly be broadcast through neuromodulatory systems, whereas a precise neuron-specific error vector must somehow be routed through the network and assigned to the correct synapses.

Several concrete implementation problems follow from this requirement for precise backward error signals. The most widely discussed is the weight transport problem: in standard backpropagation, errors are propagated backward using the transpose of the forward weight matrices, requiring the backward pathway to have access to the exact strengths of the forward synapses. Backpropagation also relies on signed error signals that may vary over many orders of magnitude, whereas biological neurons communicate through spikes and firing rates that do not naturally represent arbitrary signed derivative values. Finally, feedback connections in the brain appear to influence neural activity itself, for example through modulation, attention, or top-down prediction, while feedback in standard backpropagation carries abstract error derivatives used only for learning. These issues make a direct biological implementation of backpropagation highly problematic (Crick, 1989; Lillicrap et al., 2020).

This does not mean that backpropagation is irrelevant for neuroscience. A more careful interpretation is that strict backpropagation is biologically implausible, while backpropagation-like principles may still provide insight into how the brain solves credit assignment. Several proposed mechanisms attempt to preserve parts of the computational benefit of backpropagation while avoiding its most unrealistic requirements, for example by using random feedback weights, local activity differences, or predictive coding-style error signals. Nevertheless, these methods remain attempts to approximate or replace the exact backward gradient computation. In this thesis, we therefore focus on perturbation-based learning rules, which are less efficient than backpropagation but fit more naturally with local synaptic information, stochastic neural activity, and global scalar feedback.

2.1.3 Biologically plausible alternatives to backpropagation

The biological implausibility of strict backpropagation has motivated a broad range of alternative learning rules. These methods differ in how closely they try to approximate backpropagation and in which biological constraints they prioritize. Some aim to preserve much of the efficiency of backpropagation while relaxing specific assumptions, such as exact symmetric feedback weights. Others avoid explicit backward error propagation altogether, instead relying on local objectives, recurrent dynamics, or stochastic exploration. The purpose of this section is not to review all such methods in detail, but to briefly place perturbation-based learning within the broader landscape of biologically plausible alternatives.

Feedback alignment addresses one of the classical problems with backpropagation: the need for backward feedback weights to match the transpose of the forward weights. In standard backpropagation, the error signal in a hidden layer is computed using the downstream forward weights, which creates the weight transport problem discussed above. Feedback alignment shows that exact symmetry is not always necessary. Instead, error signals can be propagated through fixed random feedback weights, while the forward weights gradually adapt so that the updates still become useful for learning (Lillicrap et al., 2016). Direct feedback alignment extends this idea by sending error information directly from the output layer to hidden layers, bypassing parts of the backward recursive computation (Nøkland, 2016). These methods are important because they show that some of the strict requirements of backpropagation can be relaxed, although they still rely on relatively detailed vector-valued feedback signals.

Predictive coding provides another influential route toward biologically plausible credit assignment. In predictive coding models, hierarchical neural circuits attempt to predict activity in other layers, and local prediction errors are used to update synaptic weights. This idea has a long history in computational neuroscience, particularly as a model of cortical processing in the visual system (Rao and Ballard, 1999). More recent work has shown that, under certain assumptions, predictive coding networks can approximate the gradients computed by backpropagation using local Hebbian-like updates (Whittington and Bogacz, 2017). This makes predictive coding a compelling candidate for biologically plausible deep learning. At the same time, it typically requires structured recurrent dynamics and inference phases, making the connection to a concrete biological implementation more involved than for simpler scalar-feedback learning rules.

A more recent alternative is the forward-forward algorithm, which replaces the backward pass with two forward passes: one using positive data and one using negative data (Hinton, 2022). Instead of propagating gradients backward through the network, each layer is trained using a local objective that encourages high "goodness" for positive examples and low goodness for negative examples. This removes the need for a conventional backward pass and makes the learning rule more local than standard backpropagation. However, the method also changes the learning problem substantially, since it depends on the construction of positive and negative examples and layer-wise objectives rather than directly estimating the gradient of a global loss.

Perturbation methods take a different and conceptually simpler approach. Instead of computing an error derivative for each neuron or synapse, they inject random perturbations into either the weights or the neural activities and observe how these perturbations affect performance. If a perturbation improves performance, the corresponding change is reinforced; if it worsens performance, the change is reversed. This gives learning rules that combine local information about the perturbation and neural activity with a global scalar feedback signal, making them closely related to three-factor learning principles. Weight perturbation applies this idea at the level of synaptic weights, while node perturbation applies it at the level of neural activity (Williams, 1992; Werfel et al., 2005; Hiratani et al., 2022). Perturbation methods are generally less efficient than backpropagation, but they are attractive because they require only forward evaluations, stochastic fluctuations, and scalar feedback.

This thesis focuses on perturbation methods for three reasons. First, they align naturally with the biological principles introduced earlier: local synaptic information, stochastic neural activity or synaptic fluctuations, and global modulatory feedback. Second, they are simple enough to implement and analyze in standard neural network models, while still being plausible candidates for more biologically motivated architectures. Third, the literature contains fewer general theoretical comparisons between different perturbation methods than between backpropagation-like alternatives. Rather than surveying many biologically plausible algorithms superficially, this thesis therefore focuses on one family of methods in depth. The remainder of this section develops the theory of weight perturbation and node perturbation, before introducing the input-scaled node perturbation method proposed in this thesis.

2.1.4 Weight perturbation

Perturbation-based learning provides a simple alternative to backpropagation by estimating gradients using only forward evaluations of the network. Instead of explicitly computing derivatives, the idea is to introduce small random perturbations to the parameters and observe how these perturbations affect the loss. If a perturbation leads to a decrease in loss, then moving the parameters in that direction is beneficial; if it increases the loss, the direction is unfavorable. By correlating random perturbations with the resulting change in loss, one can construct a stochastic estimate of the gradient.

To formalize this perturbation-based approach, we model the network parameters as noisy rather than fixed. Let $\boldsymbol{\mu}$ denote the deterministic parameter vector that we seek to optimize, and define perturbed parameters by

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I}),$$

where $\sigma > 0$ is a small fixed perturbation scale. Rather than optimizing the deterministic loss $L(\boldsymbol{\mu})$ directly, we consider the expected loss under parameter perturbations,

$$F(\boldsymbol{\mu}) := \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}}[L(\boldsymbol{\theta})], \tag{1}$$

where $p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$. We now seek to construct an estimator of the gradient of this objective. As we will show, it is possible to obtain an unbiased estimate of $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$ using only forward evaluations of the network. For sufficiently small σ, this gradient closely approximates the gradient of the original loss $L(\boldsymbol{\mu})$.

To compute $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$, we write the expectation explicitly as

$$F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta})\, p_{\boldsymbol{\mu}}(\boldsymbol{\theta})\, d\boldsymbol{\theta},$$

and differentiate under the integral sign:

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta})\, \nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta})\, d\boldsymbol{\theta}.$$

This step is valid under standard regularity conditions, and allows us to make the dependence on $\boldsymbol{\mu}$ explicit inside the integral. Following the score-function estimator (Williams, 1992), we use the identity

$$\nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = p_{\boldsymbol{\mu}}(\boldsymbol{\theta})\, \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}),$$

which yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E}_{\boldsymbol{\theta}\sim p_{\boldsymbol{\mu}}}[L(\boldsymbol{\theta})\, \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})]\,. \tag{2}$$

This expression rewrites the gradient of an expectation as an expectation involving the score function $\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})$, enabling gradient estimation using Monte Carlo sampling.

For the Gaussian distribution $p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2\mathbf{I})$, the log-density is given by

$$\log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = -\frac{1}{2\sigma^2}\|\boldsymbol{\theta} - \boldsymbol{\mu}\|^2 + C,$$

and its gradient is

$$\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2}.$$

Substituting into (2) yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E}\left[L(\boldsymbol{\theta})\, \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2}\right].$$

Using the reparameterization $\boldsymbol{\theta} = \boldsymbol{\mu} + \sigma\boldsymbol{\xi}$, we obtain

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E}\left[L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi})\, \frac{\boldsymbol{\xi}}{\sigma}\right]. \tag{3}$$

In practice, the expectation in (3) cannot be computed exactly, as it requires integrating over all possible perturbations. Instead, we approximate it using Monte Carlo sampling, which allows us to estimate the gradient using only forward evaluations of the network. For a single sample $\boldsymbol{\xi}$, this yields the estimator

$$\widehat{g}_i = L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi})\, \frac{\xi_i}{\sigma}.$$

To reduce the variance of this estimator, one can subtract a baseline b that does not depend on the individual perturbation ξ_i. The estimator remains unbiased, since

$$\mathbb{E}\left[b\frac{\xi_i}{\sigma}\right] = 0,$$

as ξ_i has zero mean and b is independent of ξ_i. The specific batch-mean baseline used in this thesis is described in the method section.

Intuitively, the baseline centers the learning signal: perturbations that lead to lower-than-average loss reinforce their contribution, while those that perform worse than average are reversed. This reduces variance in the gradient estimate without affecting its expectation. The resulting estimator becomes

$$\widehat{g}_i = (L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b)\, \frac{\xi_i}{\sigma}. \tag{4}$$

Applying gradient descent, we update the parameters in the negative gradient direction:

$$\Delta\mu_i = -\eta\, (L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b)\, \frac{\xi_i}{\sigma}, \tag{5}$$

where η is the learning rate. This corresponds to the weight perturbation update rule. This update rule constitutes an unbiased Monte Carlo estimator of the gradient of the expected loss, implying that its expected update direction coincides with the true backpropagation gradient, although potentially with high variance.

2.1.5 Node perturbation and variants

Node perturbation provides an alternative to weight perturbation by injecting noise at the level of neuron activities rather than directly at the level of individual synaptic weights. The basic motivation is that a neural network typically contains many more weights than nodes. Perturbing the nodes therefore reduces the dimensionality of the stochastic search space, while still allowing the resulting scalar change in loss or reward to be assigned back to individual incoming weights using local presynaptic activity. In this sense, node perturbation uses a small amount of structural information about the network: a desired change in a neuron's pre-activation can be translated into a weight update by multiplying with the activity of the corresponding presynaptic neuron.

Early perturbation-based learning rules were often formulated as finite-difference methods. In Madaline-style neuron perturbation (Widrow and Lehr, 1990) and the summed weight neuron perturbation method of Flower and Jabri (1992), the change in error caused by a perturbation is used to approximate a local derivative. For a perturbation applied to neuron i, the derivative of the error with respect to the neuron's net input is approximated by a forward difference,

$$\frac{\partial E}{\partial \mathrm{net}_i} \approx \frac{\Delta E_i}{r_i},$$

where r_i is the applied perturbation. The corresponding weight update is then obtained by multiplying this local derivative estimate with the presynaptic activity. This differs from the score-function-style estimators considered in this thesis: in the finite-difference formulation, the learning signal is divided by the perturbation, whereas in score-function-style perturbation learning, the learning signal is multiplied by the perturbation.

A closely related but distinct development is the stochastic error-descent algorithm of Cauwenberghs (1993). Instead of perturbing one parameter at a time, Cauwenberghs considers a parallel random perturbation vector $\boldsymbol{\pi}$ and updates the parameters according to

$$\Delta \mathbf{p} = -\mu\big(\mathcal{E}(\mathbf{p}+\boldsymbol{\pi}) - \mathcal{E}(\mathbf{p})\big)\boldsymbol{\pi}.$$

This has the same qualitative form as modern score-function estimators: a scalar change in error is multiplied by the perturbation vector. Under small perturbations and uncorrelated perturbation components with equal variance, the expected update follows the gradient direction. This makes the method closer to the perturbation estimators studied in this thesis than the earlier finite-difference rules, although it is formulated as a general parameter perturbation method rather than specifically as node perturbation.

Modern node perturbation is usually written in a score-function-like form. For a network with pre-activation $h_k = W_k x_{k-1}$, a perturbation n_k is added to the node activity or pre-activation,

$$\tilde{h}_k = h_k + n_k, \qquad n_k \sim \mathcal{N}(0, \sigma^2 I),$$

and the resulting change in loss is used to update the incoming weights:

$$\Delta W_k = -\eta\, \Delta\ell\, \frac{n_k x_{k-1}^\top}{\sigma^2}.$$

Equivalently, if $n_k = \sigma \xi_k$ with $\xi_k \sim \mathcal{N}(0, I)$, the update is proportional to $-\Delta\ell\, \xi_k x_{k-1}^\top/\sigma$, with constant factors often absorbed into the learning rate. Here $\Delta\ell$ denotes a scalar learning signal, typically the difference between the perturbed loss and some baseline loss. This update is local in the sense that the synaptic update depends only on the presynaptic activity x_{k-1}, the postsynaptic perturbation, and a global scalar learning signal.

Different node perturbation methods mainly differ along three axes: where the perturbation is added, how the perturbation is sampled, and how the baseline is formed. In single-layer linear analyses, such as Werfel et al. (2005) and Cho et al. (2011), perturbations are added directly to the output units. In multilayer networks, more recent work such as Hiratani et al. (2022) and Dalm et al. (2024) applies perturbations to hidden-layer pre-activations or activities. This distinction is important because output perturbation is sufficient in a single-layer linear model, while multilayer networks require perturbations throughout the network if all layers are to receive credit.

Most recent theoretical treatments sample node perturbations from a Gaussian distribution. For example, Werfel et al. (2005) use Gaussian output perturbations in their linear perceptron analysis, while Hiratani et al. (2022) use Gaussian pre-activation perturbations in deep networks. Züge et al. (2023) also use Gaussian node perturbations, but in temporally extended tasks the perturbations are time-dependent, so the update contains an eligibility trace over time. In contrast, earlier finite-difference-style methods often used fixed-magnitude or random-sign perturbations, and Cauwenberghs (1993) only requires perturbations to be zero-mean, uncorrelated, and have equal variance.

The baseline used to form the scalar learning signal also varies across the literature. A common choice is a noiseless baseline, where the learning signal is the difference between the perturbed loss and the unperturbed loss,

$$\Delta\ell = \ell_{\text{pert}} - \ell_{\text{clean}}.$$

This is the form used in many standard NP and WP formulations. However, Cho et al. (2011) argue that a noiseless baseline may be biologically unrealistic because neural activity is intrinsically noisy. They therefore propose a noisy-baseline version of node perturbation, where a second independent perturbation is used to compute the baseline:

$$\Delta\ell = \ell_{\xi} - \ell_{\zeta}.$$

Hara et al. (2017) later analyze this noisy-baseline formulation using statistical mechanics. In this thesis, we instead use a batch-mean baseline, as described in the method section. This choice keeps the update within the same score-function-style family, but avoids an additional noiseless or independently perturbed baseline forward pass.

A final important distinction concerns the scale of the perturbation. In the simplest form of node perturbation, the perturbation variance is fixed globally or per layer:

$$n_k \sim \mathcal{N}(0, \sigma^2 I).$$

However, some work adjusts the perturbation scale to make comparisons between node and weight perturbation more meaningful. In the linear setting of Werfel et al. (2005), weight perturbation of MN weights induces output perturbations whose variance grows with the number of inputs N. To compare NP and WP at the same effective output perturbation strength, the node perturbation variance is therefore scaled by N, equivalently the node perturbation standard deviation is scaled by $\sqrt{N}$. Züge et al. (2023) perform a related perturbation-strength matching in temporally extended tasks, but in their formulation the weight perturbation variance is adjusted relative to an effective input dimension so that WP and NP have comparable effective output perturbation strength.

These fan-in or effective-dimension scalings are important because they recognize that the effect of a perturbation depends on the size of the layer feeding into a node. However, they remain expectation-level corrections: the scale is determined by the number of incoming units or an effective dimension, not by the actual activity vector presented to the neuron on a given sample. This distinction becomes important in the original contribution of this thesis, where the node perturbation scale is instead chosen from the pre-activation noise distribution induced by Gaussian weight perturbation.

Table 1 compares the learning rules used in different perturbation-based methods. The table shows where noise is added, how the noise is sampled, and how the weights are updated. We use σ for the noise size in all papers, even when the original paper uses another symbol. We use $\Delta\ell$ to denote the scalar global learning signal, typically computed from the difference between noisy and unperturbed performance, although the exact estimator varies between papers.

Table 1: Comparison of perturbation noise sampling in the literature.

Reference	Method(s)	Perturbed variable	Noise sampling	Notes
Hiratani et al. (2022)	Node perturbation; weight-normalized node perturbation	Pre-activation	$\sigma\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	Weight-normalized version renormalizes incoming weights.
Dalm et al. (2024)	Node perturbation; iterative node perturbation; activity-based node perturbation	Pre-activation	$\xi_l \sim \mathcal{N}(0, \sigma^2 I_l)$	Decorrelated versions exist, they use the same distribution.
Flower and Jabri (1992)	Neuron perturbation; weight perturbation; summed weight neuron perturbation	Weight / pre-activation	$\epsilon_i, \epsilon_{ij} \in \{-\sigma, \sigma\}$	Random-polarity finite-difference perturbations.
Widrow and Lehr (1990)	Madaline Rule III	Pre-activation	Small perturbation ϵ_i; no distribution specified	Early finite-difference neuron perturbation.
Cauwenberghs (1993)	Stochastic error descent	Weights	$\mathbb{E}[\epsilon_i \epsilon_j] = \sigma^2 \delta_{ij}$	Any zero-mean, uncorrelated perturbations with equal variance; simulations use $\epsilon_i = \sigma z_i,\ z_i \sim \mathrm{Unif}\{-1, 1\}$.
Werfel et al. (2005)	Node perturbation; weight perturbation	Output / weight	NP: $\boldsymbol{\epsilon} \sim \mathcal{N}(0, N\sigma^2 I_M)$; WP: $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 I_{MN})$	Node noise scaled by $\sqrt{N}$ to match the output noise induced by weight perturbation.
Züge et al. (2023)	Weight perturbation; node perturbation	Weight / pre-activation	WP: $\epsilon_{ij}^{\mathrm{WP}} \sim \mathcal{N}\left(0, \frac{\sigma_{\mathrm{eff}}^2}{a^2 N_{\mathrm{eff}}}\right)$; NP: $\epsilon_{it}^{\mathrm{NP}} \sim \mathcal{N}(0, \sigma_{\mathrm{eff}}^2)$	WP variance matched to NP output variance. N_{eff}: effective input dimension. a: error convergence factor.
Cho et al. (2011)	Node perturbation; noisy-baseline node perturbation	Output	$\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$, $\zeta_i \sim \mathcal{N}(0, \sigma_\zeta^2)$	Second perturbation replaces noiseless baseline; no layer-size scaling.
Hara et al. (2017)	Single node perturbation; double node perturbation	Output	$\epsilon_k \sim \mathcal{N}(0, \sigma_\epsilon^2)$, $\zeta_k \sim \mathcal{N}(0, \sigma_\zeta^2)$	Statistical-mechanics analysis of noisy-baseline NP; no layer-size scaling.

For compactness, let U_k denote the update direction, so that weights are updated as $W_k \leftarrow W_k - \eta_W U_k$. For decorrelated variants, $x^\star_{k-1} = R_{k-1} x_{k-1}$, and the decorrelation matrix is updated using $R_k \leftarrow R_k - \eta_R \left(x^\star_k (x^\star_k)^\top - \mathrm{diag}((x^\star_k)^2) \right) R_k$.

Table 2: Comparison of perturbation-based update rules in the literature.

Reference	Method	Perturbed variable	Noise sampling	Update rule
Hiratani et al. (2022)	Node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
Hiratani et al. (2022)	Weight-normalized node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top, \quad w_i^k \leftarrow \frac{\|w_i^k\|}{\|w_i^k - \eta_W u_i^k\|}(w_i^k - \eta_W u_i^k)$
Dalm et al. (2024)	Node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
Dalm et al. (2024)	Iterative node perturbation	Pre-activation	$v_k \sim \mathcal{N}(0, I_k)$, perturbation $h v_k$	$U_k = \sqrt{N_k} \Delta \ell \frac{v_k}{\|v\|_2^2} x_{k-1}^\top$
Dalm et al. (2024)	Activity-based node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta a_k}{\|\delta a\|_2^2} x_{k-1}^\top$
Dalm et al. (2024)	Decorrelated node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k (x_{k-1}^\star)^\top$
Dalm et al. (2024)	Decorrelated iterative node perturbation	Pre-activation	$v_k \sim \mathcal{N}(0, I_k)$, perturbation $h v_k$	$U_k = \sqrt{N_k} \Delta \ell \frac{v_k}{\|v\|_2^2} (x_{k-1}^\star)^\top$
Dalm et al. (2024)	Decorrelated activity-based node perturbation	Pre-activation	$\xi_k \sim \mathcal{N}(0, \sigma^2 I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta a_k}{\|\delta a\|_2^2} (x_{k-1}^\star)^\top$
Flower and Jabri (1992)	Neuron perturbation	Pre-activation	$\epsilon_i = \sigma z_i,\ z_i \sim \mathrm{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\epsilon_i} x_j$
Flower and Jabri (1992)	Weight perturbation	Weight	$\epsilon_{ij} = \sigma z_{ij},\ z_{ij} \sim \mathrm{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\epsilon_{ij}}$
Flower and Jabri (1992)	Summed weight neuron perturbation	Incoming weights	$\epsilon_{ij} = \sigma z_{ij},\ z_{ij} \sim \mathrm{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\epsilon_{ij}}, \quad r_i = \sum_m \epsilon_{im} x_m$
Widrow and Lehr (1990)	Madaline Rule III	Pre-activation	Small perturbation ϵ_i; no distribution specified	$U_{ij} = \frac{\Delta \ell}{\epsilon_i} x_j$
Cauwenberghs (1993)	Stochastic error descent	Weights	$\mathbb{E}[\epsilon_i \epsilon_j] = \sigma^2 \delta_{ij}$	$U = \Delta \ell\, \boldsymbol{\epsilon}$
Werfel et al. (2005)	Node perturbation	Output	$\boldsymbol{\epsilon} \sim \mathcal{N}(0, N\sigma^2 I_M)$	$U = \frac{\Delta \ell}{N\sigma^2} \boldsymbol{\epsilon} \mathbf{x}^\top$
Werfel et al. (2005)	Weight perturbation	Weight	$\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 I_{MN})$	$U = \frac{\Delta \ell}{\sigma^2} \boldsymbol{\epsilon}$
Züge et al. (2023)	Weight perturbation	Weight	$\epsilon_{ij}^{\mathrm{WP}} \sim \mathcal{N}\left(0, \frac{\sigma_{\mathrm{eff}}^2}{\alpha^2 N_{\mathrm{eff}}}\right)$	$U_{ij} = \frac{\Delta L}{\sigma_{\mathrm{WP}}^2} \epsilon_{ij}^{\mathrm{WP}}, \quad \sigma_{\mathrm{WP}}^2 = \frac{\sigma_{\mathrm{eff}}^2}{\alpha^2 N_{\mathrm{eff}}}$
Züge et al. (2023)	Node perturbation	Pre-activation	$\epsilon_{it}^{\mathrm{NP}} \sim \mathcal{N}(0, \sigma_{\mathrm{eff}}^2)$	$U_{ij} = \frac{\Delta L}{\sigma_{\mathrm{eff}}^2} \sum_{t=1}^T \epsilon_{it}^{\mathrm{NP}} x_{jt}$
Cho et al. (2011)	Node perturbation	Output	$\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$	$U_{ij} = \Delta L\, \epsilon_i x_j$
Cho et al. (2011)	Noisy-baseline node perturbation	Output	$\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$, $\zeta_i \sim \mathcal{N}(0, \sigma_\zeta^2)$	$U_{ij} = \Delta L\, \epsilon_i x_j, \quad \Delta L = L_\epsilon - L_\zeta$
Hara et al. (2017)	Single node perturbation	Output	$\epsilon_k \sim \mathcal{N}(0, \sigma_\epsilon^2)$	$U_{jk} = \frac{\Delta L}{N\sigma_\epsilon^2} \epsilon_k x_j, \quad \Delta L = L_\epsilon - L$
Hara et al. (2017)	Double node perturbation	Output	$\epsilon_k \sim \mathcal{N}(0, \sigma_\epsilon^2)$, $\zeta_k \sim \mathcal{N}(0, \sigma_\zeta^2)$	$U_{jk} = \frac{\Delta L}{N\sigma_\epsilon^2} \epsilon_k x_j, \quad \Delta L = L_\epsilon - L_\zeta$

TODO: Add an explanation of why node perturbation is also biologically plausible.

2.1.6 Existing comparisons of weight and node perturbation

Existing comparisons of weight perturbation and node perturbation are usually framed around the dimensionality of the perturbation space. In weight perturbation, noise is added to each individual weight, whereas in node perturbation, noise is added to units or pre-activations and then translated into weight updates using presynaptic activity. Since a network typically has many more weights than nodes, NP is often argued to explore a lower-dimensional stochastic space and therefore learn faster or with less noisy updates. This argument is stated explicitly by Cho et al. (2011), who note that in single-layer feedforward networks the perturbation dimensionality of WP exceeds that of NP by a factor equal to the number of input units, and therefore cite Werfel et al. (2005) for the corresponding learning-speed advantage of NP.

The main analytical support for this dimensionality argument comes from Werfel et al. (2005). They analyze online learning in a linear perceptron with N inputs and M outputs, comparing direct gradient descent, node perturbation, and weight perturbation. In this setting, NP perturbs an M-dimensional output vector, whereas WP perturbs the full MN-dimensional weight matrix. When the scalar update parameter is chosen optimally, they find that NP is slower than direct gradient descent by a factor equal to the number of output units, while WP is slower by an additional factor equal to the number of input units. This provides a precise setting in which the lower perturbation dimension of NP leads to faster learning than WP.

Later work often uses this result as the standard explanation for why NP is expected to be more efficient than WP. For example, Hiratani et al. (2022) describe WP as noisier than NP in the vanilla setting, while also noting that WP is a zeroth-order optimization method and therefore expected to scale poorly in high-dimensional parameter spaces. Dalm et al. (2024) similarly describe WP as searching a larger space because there are many more weights than nodes, which slows learning. In this literature, the comparison is therefore usually not that WP and NP estimate fundamentally different objectives, but that WP estimates the relevant update using a higher-dimensional perturbation and is therefore expected to have a worse signal-to-noise trade-off.

The dimensionality argument is not universal, however, and Züge et al. (2023) show that WP can perform similarly to or better than NP in temporally extended tasks. Their key observation is that temporal structure changes the effective perturbation dimensions: WP perturbs the weights once per trial, while standard NP must perturb node inputs over time in order to explore time-dependent trajectories. In low-dimensional temporal tasks, WP-induced output perturbations are constrained to realizable trajectories, whereas NP may inject arbitrary time-dependent perturbations that contain components irrelevant or impossible for the network to realize. Their analysis and simulations therefore show that task structure can reverse the usual preference for NP, making WP a competitive baseline rather than merely an inefficient alternative.

2.1.7 Literature gap

The existing literature leaves the relationship between WP and NP somewhat unresolved. On the one hand, NP is often presented as the more efficient method because it perturbs a lower-dimensional space: nodes rather than weights. This intuition is supported by the linear perceptron analysis of Werfel et al. (2005), where NP learns faster than WP by a factor related to the number of input units. On the other hand, this conclusion is not universal. Züge et al. (2023) show that in temporally extended tasks, WP can perform similarly to or better than NP, because temporal structure changes the effective perturbation dimensions. Thus, the literature contains both a standard argument for why NP should be better and important examples where that argument breaks down.

What is missing is a more direct theoretical comparison of WP and NP in the nonlinear multilayer setting used by modern neural networks. Existing comparisons often rely on counting perturbation dimensions, analyzing simplified linear models, or studying specific task structures. These results

are valuable, but they do not fully explain whether the apparent advantage of NP comes only from perturbing fewer variables, or whether there is a deeper estimator-level relationship between node-level and weight-level perturbations. This motivates the analysis in the next section, where we compare WP and NP by first asking what kind of node-level noise is actually induced when the weights of a nonlinear multilayer perceptron are perturbed.

This literature review establishes the current state of perturbation-based learning and motivates the theoretical contribution developed next.

2.2 Original contribution to theory development

2.2.1 Overview of the contribution

The literature reviewed above leaves the relationship between weight perturbation and node perturbation only partially resolved. Existing work suggests that NP is often preferable because it perturbs a lower-dimensional space, but this argument is not a direct comparison of the two estimators in nonlinear multilayer networks. To make this comparison more precise, we start from WP and ask what effect Gaussian weight perturbations have at the level of the pre-activations.

This leads to three theoretical contributions. First, we derive the conditional distribution of the pre-activation noise induced by Gaussian WP in a nonlinear multilayer perceptron. Second, we use this induced noise distribution to define input-scaled node perturbation (IS-NP), a node perturbation method whose perturbation scale depends on the norm of the incoming activation vector. Third, we show that IS-NP matches WP in expectation but has lower or equal variance, because the IS-NP estimator can be viewed as a conditioned version of the WP estimator.

Together, these results provide a direct theoretical reason to prefer IS-NP over WP when the goal is to estimate the same expected update with lower variance. The comparison with existing NP variants is different: the theory suggests that IS-NP should be better scaled than fixed-variance NP and fan-in-scaled NP, but it does not prove that IS-NP has lower variance or better learning dynamics than those methods. This motivates the empirical evaluation later in the thesis, where IS-NP is compared against WP, vanilla NP, and fan-in-scaled NP.

2.2.2 Pre-activation noise induced by WP

The first step in comparing WP and NP is to express the effect of WP at the same level where NP applies perturbations: the pre-activations. This allows us to ask whether a node-level perturbation rule can reproduce the effect of weight perturbation without sampling all individual weight perturbations.

Recall that in the previous section, we modeled the parameters as $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$. For neural networks, this corresponds to perturbing each weight independently as

$$\mathbf{W}^{\ell\prime} = \mathbf{W}^{\ell} + \sigma \mathbf{E}^{\ell}, \qquad E_{ij}^{\ell} \sim \mathcal{N}(0,1) \text{ i.i.d.}$$

The pre-activation at neuron j in layer ℓ is given by

$$z_j^{\ell} = \sum_i W_{ij}^{\ell} x_i^{\ell-1}.$$

After perturbation, this becomes

$$z_j^{\ell\prime} = \sum_i W_{ij}^{\ell\prime} x_i^{\ell-1\prime} = \sum_i (W_{ij}^{\ell} + \sigma E_{ij}^{\ell})\, x_i^{\ell-1\prime}$$

This expression separates into a deterministic component and a stochastic component induced by the weight perturbations:

$$z_j^{\ell\prime} = \sum_i W_{ij}^{\ell} x_i^{\ell-1\prime} + \sigma \sum_i E_{ij}^{\ell} x_i^{\ell-1\prime}.$$

Define the induced pre-activation noise as

$$n_j^\ell := \sigma \sum_i E_{ij}^\ell x_i^{\ell-1\prime}. \tag{6}$$

We can now write the full pre-activation in compact form as

$$z_j^{\ell\prime} = (\mathbf{W}^\ell \mathbf{x}^{\ell-1\prime})_j + n_j^\ell,$$

where

$$(\mathbf{W}^\ell \mathbf{x}^{\ell-1\prime})_j = \sum_i W_{ij}^\ell x_i^{\ell-1\prime}.$$

Now condition on the incoming perturbed activations $\mathbf{x}^{\ell-1\prime}$. Under this conditioning, the coefficients $x_i^{\ell-1\prime}$ are fixed, while E_{ij}^ℓ remain independent standard Gaussians. From (6), it follows that n_j^ℓ is a linear combination of independent Gaussian variables, and therefore Gaussian with

$$n_j^\ell \mid \mathbf{x}^{\ell-1\prime} \sim \mathcal{N}\left(0, \sigma^2 \sum_i (x_i^{\ell-1\prime})^2\right) = \mathcal{N}\big(0, \sigma^2 \|\mathbf{x}^{\ell-1\prime}\|^2\big)\,. \tag{7}$$

Thus, weight perturbation induces Gaussian noise at the pre-activations, with variance proportional to the squared norm of the incoming activations. In other words, the effect of perturbing the weights can be fully characterized as injecting structured Gaussian noise directly into the pre-activations.

This shows that Gaussian WP does not induce arbitrary node noise. It induces node noise whose variance depends on the squared norm of the incoming activation vector. This observation is the basis for the IS-NP rule introduced next.

2.2.3 Input-scaled node perturbation

The induced pre-activation noise derived above suggests a natural modification of standard node perturbation. In standard node perturbation, noise is injected directly at the level of the pre-activations, typically using a fixed perturbation scale,

$$z_j^{\ell\prime} = z_j^\ell + \epsilon_j^\ell, \qquad \epsilon_j^\ell \sim \mathcal{N}(0, \sigma^2),$$

and a gradient estimate is constructed from the resulting change in loss (Hiratani et al., 2022). This fixed-scale formulation treats the perturbation scale as a global or layerwise hyperparameter, independent of the actual activity vector feeding into the neuron.

A related variant is fan-in-scaled node perturbation, where the perturbation scale is adjusted according to the number of incoming units. For a layer with input dimension $d_{\ell-1}$, this corresponds to sampling noise with standard deviation proportional to $\sqrt{d_{\ell-1}}$, or $\sqrt{d_{\ell-1}+1}$ if the bias is included as an additional input. This accounts for the fact that, under simplifying assumptions, the norm of the incoming activation vector grows with the square root of the layer width. However, this remains an expectation-level correction: the scale depends on the size of the layer, not on the actual incoming activation vector for a given sample.

The derivation in the previous section motivates a more direct scaling rule. Since Gaussian weight perturbation induces pre-activation noise with variance proportional to $\|\mathbf{x}^{\ell-1\prime}\|^2$, we define input-scaled node perturbation (IS-NP) by sampling node noise from this induced distribution:

$$z^{\ell\prime} = \mathbf{W}^\ell \mathbf{x}^{\ell-1\prime} + \mathbf{n}^\ell, \qquad n_j^\ell \mid \mathbf{x}^{\ell-1\prime} \sim \mathcal{N}\big(0, \sigma^2 \|\mathbf{x}^{\ell-1\prime}\|^2\big)\,.$$

Thus, unlike vanilla NP, the perturbation scale is not fixed. Unlike fan-in-scaled NP, it is not determined only by the number of incoming units. Instead, IS-NP scales the perturbation by the actual norm of the incoming activation vector at each layer and for each input example.

We first consider the effect of this perturbation at the level of the pre-activations. Since

$$n_j^\ell \mid \mathbf{x}^{\ell-1\prime} \sim \mathcal{N}\big(0, \sigma^2 \|\mathbf{x}^{\ell-1\prime}\|^2\big)\,,$$

the corresponding score-function term for the pre-activation perturbation is proportional to

$$\frac{n_j^\ell}{\sigma^2 \|\mathbf{x}^{\ell-1\prime}\|^2}.$$

The resulting gradient estimate with respect to z_j^ℓ is therefore

$$\widehat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b)\, \frac{n_j^\ell}{\sigma^2 \|\mathbf{x}^{\ell-1\prime}\|^2}.$$

Using the chain rule,

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1\prime},$$

we obtain the corresponding estimator for the weights:

$$\widehat{g}_{ij,\ell}^{\text{IS-NP}} = (L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b)\, \frac{n_j^\ell x_i^{\ell-1\prime}}{\sigma^2 \|\mathbf{x}^{\ell-1\prime}\|^2}. \tag{8}$$

Applying gradient descent, the update rule becomes

$$\Delta W_{ij}^\ell = -\eta\, (L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b)\, \frac{n_j^\ell x_i^{\ell-1\prime}}{\sigma^2 \|\mathbf{x}^{\ell-1\prime}\|^2}. \tag{9}$$

Intuitively, IS-NP injects noise at the pre-activations with a scale matched to the effect that weight perturbation would have produced at that neuron. The resulting scalar learning signal is then distributed across the incoming weights in proportion to the corresponding presynaptic activities. Large incoming activations therefore receive a larger share of the update, while small incoming activations receive a smaller share.

This construction makes IS-NP different from both vanilla NP and fan-in-scaled NP. Vanilla NP uses the same perturbation scale regardless of the input activity. Fan-in-scaled NP corrects this scale only through the width of the incoming layer. IS-NP instead uses the actual incoming activation norm, making the node perturbation scale sample-dependent and layer-dependent. The next section shows that this choice is not only intuitively motivated, but also makes IS-NP directly comparable to WP in expectation.

2.2.4 Expected equivalence between WP and IS-NP

Before comparing the variance of WP and IS-NP, we first show that the two estimators are matched in expectation. This is important because a variance comparison is only meaningful if the estimators target the same expected update. The key observation is that IS-NP can be obtained by conditioning the WP estimator on the pre-activation noise induced by the weight perturbations.

Recall that the WP gradient estimator for weight W_{ij}^ℓ can be written as

$$\widehat{g}_{ij,\ell}^{\text{WP}} = (L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b)\, \frac{E_{ij}^\ell}{\sigma}, \tag{10}$$

where $E_{ij}^\ell \sim \mathcal{N}(0,1)$ is the standard Gaussian perturbation applied to the corresponding weight. Let $N = (n^1, \ldots, n^L)$ denote the full field of pre-activation noises induced by WP. Conditioned on N, the perturbed forward trajectory is fixed, since the network evolves according to

$$z^{\ell\prime} = \mathbf{W}^\ell \mathbf{x}^{\ell-1\prime} + \mathbf{n}^\ell.$$

Therefore, the loss difference $L(\boldsymbol{\mu}+\sigma\boldsymbol{\xi})-b$ and the activations $\mathbf{x}^{\ell-1\prime}$ are deterministic functions of N.

We now compute the conditional expectation of the WP estimator given the induced pre-activation noise. From Eq. (6),

$$n_j^\ell = \sigma \sum_i E_{ij}^\ell x_i^{\ell-1\prime}.$$

Since the vector $E_{:j}^\ell$ is Gaussian, standard Gaussian conditioning gives

$$\mathbb{E}\big[E_{ij}^\ell \mid N\big] = \frac{x_i^{\ell-1\prime}}{\sigma\|\mathbf{x}^{\ell-1\prime}\|^2} n_j^\ell.$$

Substituting this into Eq. (10) gives

$$\mathbb{E}\big[\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N\big] = (L(\boldsymbol{\mu}+\sigma\boldsymbol{\xi})-b)\,\frac{1}{\sigma}\mathbb{E}\big[E_{ij}^\ell \mid N\big]\,.$$

Hence,

$$\mathbb{E}\big[\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N\big] = (L(\boldsymbol{\mu}+\sigma\boldsymbol{\xi})-b)\,\frac{n_j^\ell x_i^{\ell-1\prime}}{\sigma^2\|\mathbf{x}^{\ell-1\prime}\|^2}.$$

By Eq. (8), this is exactly the IS-NP estimator:

$$\mathbb{E}\big[\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N\big] = \widehat{g}_{ij,\ell}^{\text{IS-NP}}. \tag{11}$$

Taking expectation on both sides and applying the tower property gives

$$\mathbb{E}\big[\widehat{g}_{ij,\ell}^{\mathrm{WP}}\big] = \mathbb{E}\big[\mathbb{E}\big[\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N\big]\big] = \mathbb{E}\big[\widehat{g}_{ij,\ell}^{\text{IS-NP}}\big]\,.$$

Thus, WP and IS-NP produce the same expected gradient estimate for each weight. The difference between them is not the expected update direction, but the amount of stochastic variability around that expectation. This is the basis for the variance comparison in the next section.

2.2.5 Variance reduction theorem

The previous section showed that IS-NP and WP are matched in expectation. We now show that they differ in variance. More specifically, IS-NP can be viewed as the conditional expectation of WP given the induced pre-activation noise field. This means that IS-NP removes weight-noise variability that affects the WP update but does not affect the actual perturbed forward pass. The result is a direct variance reduction by the law of total variance.

Proposition 1. *For Gaussian weight perturbations and the corresponding input-scaled node perturbation estimator defined in Eq.* (8)*, the coordinate-wise variance of IS-NP is no larger than that of WP:*

$$\mathrm{Var}\big(\widehat{g}_{ij,\ell}^{\text{IS-NP}}\big) \le \mathrm{Var}\big(\widehat{g}_{ij,\ell}^{\mathrm{WP}}\big)\,.$$

Proof. We consider the weight perturbation estimator in (10), and analyze its variance by conditioning on the induced pre-activation noise. Recall from (6) that n_j^ℓ is a linear combination of the weight perturbations.

Condition on the full induced noise field $N = (n^1, \ldots, n^L)$. Since the perturbed forward pass satisfies

$$z^{\ell\prime} = W^\ell x^{\ell-1\prime} + n^\ell,$$

the entire perturbed trajectory $(x^{1\prime}, \ldots, x^{L\prime})$ is uniquely determined by N. Hence both the learning signal $\delta L(N)$ and all activations $x^{\ell\prime}$ are deterministic given N.

Now fix (i, j, ℓ). From (6), and since $E_{:j}^\ell$ is Gaussian, standard Gaussian regression gives

$$\mathbb{E}[E_{ij}^\ell \mid N] = \frac{x_i^{\ell-1\prime}}{\sigma\|x^{\ell-1\prime}\|^2}\, n_j^\ell.$$

Therefore,

$$\mathbb{E}[\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N] = \frac{\delta L(N)}{\sigma}\,\mathbb{E}[E_{ij}^{\ell} \mid N] = \delta L(N)\,\frac{n_j^{\ell} x_i^{\ell-1\prime}}{\sigma^2 \|x^{\ell-1\prime}\|^2} = \widehat{g}_{ij,\ell}^{\mathrm{IS\text{-}NP}}.$$

Applying the law of total variance,

$$\mathrm{Var}(\widehat{g}_{ij,\ell}^{\mathrm{WP}}) = \mathrm{Var}\big(\mathbb{E}[\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N]\big) + \mathbb{E}\big[\mathrm{Var}(\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N)\big].$$

Using the identity above,

$$\mathrm{Var}(\widehat{g}_{ij,\ell}^{\mathrm{WP}}) = \mathrm{Var}(\widehat{g}_{ij,\ell}^{\mathrm{IS\text{-}NP}}) + \mathbb{E}\big[\mathrm{Var}(\widehat{g}_{ij,\ell}^{\mathrm{WP}} \mid N)\big].$$

Since the second term is nonnegative, it follows that

$$\mathrm{Var}(\widehat{g}_{ij,\ell}^{\mathrm{IS\text{-}NP}}) \le \mathrm{Var}(\widehat{g}_{ij,\ell}^{\mathrm{WP}}).$$ □

This result gives a direct theoretical comparison between WP and IS-NP. Both estimators produce the same expected update, but WP contains additional randomness from the individual weight perturbations that is irrelevant once the induced pre-activation noise is fixed. In contrast, IS-NP samples directly at the pre-activation level and therefore removes this redundant variability.

Intuitively, many different weight-noise realizations can produce the same pre-activation noise at a neuron. Once the pre-activation noise field N is fixed, the perturbed activations and the scalar learning signal are already determined. Any remaining variation in the individual weight perturbations changes the WP update without changing the network trajectory or the loss. This extra randomness contributes to the variance of WP but not to IS-NP.

Thus, IS-NP can be interpreted as a Rao–Blackwellized version of WP. It preserves the expected update while reducing the estimator variance. This provides a theoretical reason to prefer IS-NP over WP when the goal is to estimate the same perturbation-based learning signal with less stochastic noise. The next section discusses how this input-scaled construction relates to existing node perturbation variants, where the comparison is motivated by scaling considerations rather than by the variance theorem above.

2.2.6 Why IS-NP may improve over existing NP variants

The variance result above gives a theoretical reason to prefer IS-NP over WP, but it does not by itself prove that IS-NP is better than other node perturbation methods. Vanilla NP and fan-in-scaled NP use different perturbation distributions, and are therefore not simply higher-variance versions of the exact same estimator. However, the induced-noise derivation gives a clear reason to expect IS-NP to be better scaled than both of these existing NP variants.

In vanilla NP, the pre-activation noise is sampled with a fixed variance,

$$\epsilon_j^{\ell} \sim \mathcal{N}(0, \sigma^2),$$

independent of the activity vector feeding into the neuron. This means that the same absolute perturbation scale is used whether the incoming activation norm is large or small. From the perspective of estimating weight-level gradients, this is potentially problematic. If $\|\mathbf{x}^{\ell-1}\|$ is small, fixed-variance NP may create a large change in the pre-activation even though a comparable weight perturbation would have had very little effect. Conversely, if $\|\mathbf{x}^{\ell-1}\|$ is large, the same fixed node perturbation may be too small relative to the perturbation that weight noise would naturally induce. In both cases, the perturbation scale is mismatched to the sensitivity of the pre-activation to its incoming weights.

Fan-in-scaled NP partially addresses this issue by scaling the perturbation according to the number of incoming units. If the incoming activations are approximately independent and similarly distributed, then the expected squared norm of the incoming activity vector grows with the layer width. In that case, using a perturbation scale proportional to $\sqrt{d_{\ell-1}}$, or $\sqrt{d_{\ell-1}+1}$ when including the bias, can be interpreted as an average correction for the size of the incoming layer. This

makes fan-in scaling more closely aligned with the perturbation scale induced by WP than vanilla NP.

However, fan-in scaling is still only an expectation-level correction. It assumes that the norm of the incoming activation vector is well represented by the number of incoming units, which need not hold for individual samples, different layers, or unnormalised inputs. Two activation vectors with the same dimension can have very different norms, and therefore a weight perturbation would induce very different pre-activation noise in the two cases. Fan-in-scaled NP ignores this sample-wise variation.

IS-NP instead scales the perturbation using the actual incoming activation norm,

$$n_j^\ell \mid \mathbf{x}^{\ell-1} \sim \mathcal{N}\big(0, \sigma^2 \|\mathbf{x}^{\ell-1}\|^2\big) .$$

This makes the perturbation scale both layer-dependent and sample-dependent. When the incoming activation norm is small, IS-NP applies smaller node perturbations; when the norm is large, it applies larger perturbations. In this sense, IS-NP can be viewed as a sample-wise version of the correction that fan-in scaling only approximates on average.

This gives a principled reason to expect IS-NP to improve over vanilla NP and fan-in-scaled NP, especially when activation norms vary substantially across samples or layers. Nevertheless, this is not a formal dominance result. The variance theorem compares IS-NP to WP under a matched perturbation construction, while the comparison to vanilla and fan-in-scaled NP depends on how the different perturbation scales affect learning in practice. The empirical section therefore tests whether the improved scaling of IS-NP leads to better directional alignment, lower estimator variance, and improved training performance relative to existing NP variants.

2.2.7 Theoretical summary and empirical hypotheses

The analysis above establishes IS-NP as a variance-reduced version of WP. Starting from Gaussian weight perturbation, we derived the pre-activation noise induced by perturbing the weights and used this distribution to define IS-NP. This construction makes IS-NP matched to WP in expectation, while the variance reduction theorem shows that IS-NP has coordinate-wise variance no larger than WP. This is a useful theoretical contribution in itself: instead of relying only on the usual dimensionality argument that NP perturbs fewer variables than WP, we obtain a direct estimator-level comparison showing that IS-NP preserves the expected WP update while removing redundant weight-level randomness.

The comparison with existing NP variants is more empirical. The theorem does not prove that IS-NP must outperform vanilla NP or fan-in-scaled NP, since these methods use different perturbation distributions rather than being unconditioned versions of the same estimator. However, the induced-noise derivation suggests that IS-NP should be better scaled: vanilla NP uses a fixed perturbation scale, fan-in-scaled NP corrects only for layer width, while IS-NP uses the actual incoming activation norm for each sample and layer. The empirical section therefore tests whether IS-NP improves over WP, vanilla NP, and fan-in-scaled NP in terms of learning curves, directional alignment with the backpropagation gradient, estimator variance, and stability.

3 Method

3.1 Empirical evaluation strategy

The empirical evaluation is designed to answer two related questions. First, how well do learning methods based on perturbations work in practice as alternatives to backpropagation? Second, does the proposed input-scaled node perturbation (IS-NP) method improve over existing perturbation methods in the way suggested by the theory developed in Chapter 2?

The first question concerns the overall practical viability of perturbation-based learning. Weight

perturbation and node perturbation are attractive because they estimate useful update directions using only forward perturbations and scalar learning signals, but this comes at the cost of noisy gradient estimates. The experiments therefore assess not only whether these methods can learn simple tasks, but also how their performance changes as the learning problem becomes more difficult. In this sense, poor performance on harder tasks is still informative, because it helps identify the limits of perturbation-based learning.

The second question concerns the specific contribution of this thesis. The theory chapter showed that IS-NP is matched to WP in expectation while having lower coordinate-wise estimator variance. It also motivated why IS-NP may be better scaled than vanilla NP and fan-in-scaled NP, although this latter comparison was not proven theoretically. The empirical evaluation therefore compares IS-NP against WP, vanilla NP, and fan-in-scaled NP to test whether these theoretical and intuitive advantages lead to better learning dynamics in practice.

To answer these questions, the methods are evaluated on four supervised learning problems of increasing complexity and compared against backpropagation. Backpropagation is included as the standard reference method, not because the perturbation methods are expected to outperform it, but because it shows how well the same architecture can learn when exact gradients are available. The task progression from synthetic sinus regression to California housing, MNIST, and CIFAR-10 is chosen to reveal both successful learning and failure modes. Easier tasks show whether the methods work under favourable conditions, while harder tasks make it possible to see where the methods break down and whether the differences between perturbation rules become more pronounced.

All experiments use fully connected multilayer perceptrons to keep the comparison focused on the learning rule. For image classification tasks such as MNIST and CIFAR-10, convolutional networks would likely achieve higher absolute accuracy. However, the goal is not to maximize benchmark performance, but to compare how different learning rules behave under the same model class. Using MLPs throughout therefore makes the empirical comparison more controlled, even if it reduces absolute performance on image tasks.

3.2 Evaluation metrics

The primary measure of performance is the evolution of training and test loss. These quantities determine whether a learning rule successfully optimizes the task and whether the resulting model generalizes to unseen data. However, loss curves alone provide limited diagnostic information. If a perturbation-based method fails to learn, the loss curve shows the failure, but not whether it was caused by poor update directions, excessive estimator noise, or unstable learning dynamics.

To understand the learning curves, we therefore evaluate diagnostic metrics that capture different properties of the stochastic gradient estimators. Prior work on node perturbation, particularly Hiratani et al. (2022), highlights directional alignment, estimator variance, and stability as central quantities for understanding perturbation-based learning. We follow the same general perspective, but focus on empirical measurements rather than deriving full analytical expressions for each method. The following sections describe the metrics used to evaluate both final performance and estimator quality.

3.2.1 Training and test performance

Training loss measures how well the learning rule reduces the objective on the data used for optimization. Test loss measures how well the resulting model performs on held-out data that was not used during training. Together, these metrics capture the central purpose of supervised learning: fitting the structure in the training data while learning a function that generalizes to new inputs. They are therefore the most important empirical measures in this thesis. Directional alignment and estimator variance are useful because they help explain the learning curves, but training and test performance determine whether a learning rule is practically successful.

In all tasks, the optimization objective is mean squared error (MSE). For the regression tasks, the training and test curves report MSE, and R^2 is used as an additional score in the summary table. For MNIST and CIFAR-10, the networks are also trained with MSE, using one-hot class targets, but performance is additionally measured by classification accuracy. Accuracy is computed by taking the largest output component as the predicted class and comparing it with the true label. After each training epoch, the model is evaluated without perturbation noise on a training-evaluation set and on the test set. The reported loss is the mean MSE over the evaluated examples. The final summary table reports the lowest recorded training and test MSE, the highest recorded training and test score, and the convergence epoch. The convergence epoch is defined as the first epoch at which the test loss has achieved 90% of the total improvement from the initial test loss to the best test loss observed during the run. These quantities are computed after selecting the hyperparameters for each method through the search procedure described in Section 3.5.

3.2.2 Directional alignment

Directional alignment measures how well the gradient estimate produced by a perturbation-based learning rule points in the same direction as the true gradient of the loss. Intuitively, if the estimated update direction is well aligned with the true gradient, the learning rule is likely to reduce the loss efficiently. Conversely, if the estimated gradient frequently points in incorrect directions, learning becomes inefficient and may even increase the loss. A natural way to quantify this alignment is through the cosine similarity between the estimated gradient $\widehat{g}$ and the true gradient g:

$$\cos(\widehat{g}, g) = \frac{\widehat{g}^\top g}{\|\widehat{g}\|\|g\|}.$$

Here, $\widehat{g}$ denotes the gradient estimator produced by the learning rule, while $g = \nabla_\theta L(\theta)$ denotes the true gradient, computed using backpropagation in the experiments. The metric takes values in $[-1, 1]$, where 1 indicates perfect alignment, 0 indicates orthogonality, and negative values indicate that the estimated update direction points against the true descent direction.

The relevance of directional alignment can be seen from a first-order approximation of the loss change. Let $L(\theta)$ be a differentiable loss function and consider an update

$$\theta^+ = \theta + \Delta\theta, \qquad \Delta\theta = -\eta\widehat{g},$$

where $\eta > 0$ is the learning rate. A first-order Taylor expansion around θ gives

$$L(\theta^+) = L(\theta + \Delta\theta) \approx L(\theta) + \nabla_\theta L(\theta)^\top \Delta\theta.$$

Writing $g = \nabla_\theta L(\theta)$ and substituting $\Delta\theta = -\eta\widehat{g}$, we obtain

$$\Delta L = L(\theta^+) - L(\theta) \approx -\eta g^\top \widehat{g}.$$

Using

$$g^\top \widehat{g} = \|g\|\|\widehat{g}\| \cos(\widehat{g}, g),$$

this becomes

$$\Delta L \approx -\eta\|g\|\|\widehat{g}\| \cos(\widehat{g}, g).$$

Thus, for fixed gradient norms, a higher cosine similarity gives a larger first-order decrease in the loss. Positive cosine similarity corresponds to a locally descending update, zero cosine similarity gives no first-order improvement, and negative cosine similarity corresponds to local ascent. This makes cosine similarity a useful diagnostic for the directional quality of perturbation-based gradient estimates, although the actual loss change also depends on update magnitude, estimator variance, and curvature.

In practice, cosine similarity is measured on frozen backpropagation checkpoints rather than during the final training runs. For each task, a model is first trained with backpropagation and saved at three checkpoints representing early, middle, and later training; the checkpoint epochs are given

in Table 5. At each checkpoint and for each mini-batch in the training set, the reference direction is the exact backpropagation descent direction

$$u_{\mathrm{BP}} = -\nabla_\theta L_{\mathcal{B}}(\theta),$$

flattened over all weights and biases. For each perturbation method, the same frozen parameters and mini-batch are used to draw K independent perturbation estimates, where K is the task-specific number of perturbations in Table 5. Each estimate is stored as the raw parameter-update vector before multiplication by the learning rate. The K estimates are averaged,

$$\bar{u}_m = \frac{1}{K}\sum_{k=1}^{K} \widehat{u}_m^{(k)},$$

and the reported batch-level cosine is

$$\cos(\bar{u}_m, u_{\mathrm{BP}}) = \frac{\bar{u}_m^\top u_{\mathrm{BP}}}{\|\bar{u}_m\|\|u_{\mathrm{BP}}\|}.$$

The final cosine value for a method is obtained by averaging this all-parameter cosine over all analysis mini-batches and over the three frozen checkpoints. This procedure ensures that WP, IS-NP, vanilla NP, and fan-in-scaled NP are compared on exactly the same weights and data batches. Backpropagation itself is assigned cosine 1 in the summary table, since it defines the reference direction.

3.2.3 Estimator variance

Directional alignment measures whether a gradient estimator points in a useful direction, but it does not capture how much the estimate fluctuates between perturbation samples. In perturbation-based learning, the gradient estimate is obtained from random perturbations, so two perturbation samples at the same parameter values can produce different update directions. Even if the estimator is well aligned with the true gradient on average, large fluctuations can make learning inefficient by requiring smaller learning rates or more samples. We therefore measure the variance of the gradient estimator as a diagnostic of update noise.

For a vector-valued gradient estimator $\widehat{g}$, estimator variability is naturally described by the covariance matrix

$$\Sigma = \mathbb{E}\big[(\widehat{g} - \mathbb{E}[\widehat{g}])(\widehat{g} - \mathbb{E}[\widehat{g}])^\top\big].$$

When the estimator is unbiased for the gradient $g = \nabla_\theta L(\theta)$, this can be written as

$$\Sigma = \mathbb{E}\big[(\widehat{g} - g)(\widehat{g} - g)^\top\big].$$

The diagonal entries of Σ describe the variance of individual gradient components, while the off-diagonal entries describe correlations between fluctuations in different parameters. In practice, this covariance cannot be computed exactly and is instead estimated from multiple perturbation-based gradient estimates sampled at the same parameter values.

The relevance of estimator variance can be seen from a second-order approximation of the loss change. Let $L(\theta)$ be twice differentiable and consider the update

$$\theta^+ = \theta + \Delta\theta, \qquad \Delta\theta = -\eta\widehat{g},$$

where $\eta > 0$ is the learning rate. A second-order Taylor expansion around θ gives

$$L(\theta^+) \approx L(\theta) + \nabla_\theta L(\theta)^\top \Delta\theta + \frac{1}{2}\Delta\theta^\top H \Delta\theta,$$

where $H = \nabla_\theta^2 L(\theta)$ is the Hessian. Writing $g = \nabla_\theta L(\theta)$ and substituting $\Delta\theta = -\eta\widehat{g}$, we obtain

$$\Delta L = L(\theta^+) - L(\theta) \approx -\eta g^\top \widehat{g} + \frac{\eta^2}{2}\widehat{g}^\top H \widehat{g}.$$

Taking expectation over the stochastic gradient estimator, and assuming $\mathbb{E}[\widehat{g}] = g$, gives

$$\mathbb{E}[\Delta L] \approx -\eta\|g\|^2 + \frac{\eta^2}{2} g^\top H g + \frac{\eta^2}{2} \operatorname{tr}(H\Sigma).$$

The first term is the desired first-order decrease in loss, while the second term is the curvature cost of taking a finite step in the true gradient direction. The final term is the additional cost caused by estimator variance. This shows that gradient noise is especially harmful when it lies in high-curvature directions of the loss landscape, since its effect is weighted by H through $\operatorname{tr}(H\Sigma)$.

Estimator variance is therefore a useful diagnostic for perturbation-based learning. High variance means that individual updates may deviate substantially from the average update direction, even when the estimator is unbiased or well aligned on average. This is particularly relevant for the comparison between WP and IS-NP, since the theory in Section 2.2 predicts that IS-NP should preserve the expected WP update while reducing coordinate-wise estimator variance.

The empirical variance measurement uses the same frozen backpropagation checkpoints and perturbation samples as the cosine-similarity measurement. For each checkpoint, mini-batch, and perturbation method, the exact backpropagation descent direction u_{BP} is computed and flattened over all weights and biases. We then draw K independent perturbation-based update estimates $\widehat{u}^{(1)}, \ldots, \widehat{u}^{(K)}$ at the same frozen parameters and on the same mini-batch. Since the perturbation estimators used here are unbiased estimators of the backpropagation update direction, subtracting u_{BP} gives an empirical estimate of estimator variance rather than merely an arbitrary mean squared error to a reference vector. For P parameters, the measured variance for one method, checkpoint, and mini-batch is

$$\widehat{V} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{P} \left\| \widehat{u}^{(k)} - u_{\mathrm{BP}} \right\|_2^2.$$

Equivalently, under unbiasedness this estimates the coordinate-averaged trace of the estimator covariance,

$$\mathbb{E}[\widehat{V}] = \frac{1}{P} \operatorname{tr}(\Sigma).$$

The reported variance for each method is obtained by averaging this quantity over all analysis mini-batches and over the three frozen checkpoints. As with cosine similarity, the final reported value uses the all-parameter vector rather than separate layerwise values. Backpropagation is assigned variance 0 in the summary table because it defines the deterministic reference update.

3.2.4 Stability / update magnitude diagnostics

3.3 Learning problems and datasets

The experiments use four supervised learning problems: two regression tasks and two classification tasks. All models are fully connected multilayer perceptrons (MLPs), and all methods are evaluated on the same data splits and network architecture within each task. This keeps the comparison focused on the learning rule rather than on differences in model class. For the regression tasks, performance is measured using mean squared error (MSE) and R^2. For the classification tasks, the network output is a 10-dimensional vector trained against one-hot labels using MSE, while classification accuracy is computed from the largest output component.

3.3.1 Synthetic sinus regression

The sinus task is a synthetic regression problem designed to provide a controlled, low-dimensional setting where the target function is nonlinear but smooth. A latent input vector $\mathbf{u} \in [-1, 1]^8$ is sampled uniformly, and the target is

$$y = \frac{0.6 \sum_i \sin(\pi u_i) + 0.3 \sum_i u_i^2 + 0.2 u_1 u_2}{\sqrt{8}}.$$

Training targets are corrupted with additive Gaussian noise with standard deviation 0.1, while the test targets are noiseless. In the default sinus experiment, the model input is equal to the latent vector, $\mathbf{x} = \mathbf{u}$, so the inputs lie in $[-1, 1]^8$.

3.3.2 California housing regression

The California housing task is a real tabular regression problem. Each input contains eight housing and geographical features, and the target is the median house value. The dataset is split into training and test sets using an 80/20 split. Both input features and targets are standardized using the mean and standard deviation computed on the training split. This makes the MSE values comparable across training and test splits and prevents differences in feature scale from dominating the perturbation dynamics.

3.3.3 MNIST classification

MNIST is used as a simple image classification task with ten digit classes. The 28×28 grayscale images are converted to floating point values in $[0, 1]$, mean-centered using the scalar mean of the training subset, and flattened into 784-dimensional input vectors. A random subset of 20,000 training images and 5,000 test images is used. Labels are represented as one-hot vectors for the MSE training objective, while accuracy is computed using the class with the largest model output.

3.3.4 CIFAR-10 classification

CIFAR-10 is included as a more difficult image classification task with higher-dimensional inputs and more complex visual structure. The 32×32 RGB images are converted to floating point values in $[0, 1]$, mean-centered using the scalar mean of the training subset, rearranged to channel-first order, and flattened into 3072-dimensional input vectors. As for MNIST, a random subset of 20,000 training images and 5,000 test images is used, with one-hot targets for MSE training and argmax accuracy for evaluation.

3.3.5 Scaled-input diagnostics

In addition to the four main task settings, scaled-input diagnostics are run for sinus regression and MNIST. For sinus, the latent target remains defined on $\mathbf{u} \in [-1, 1]^8$, but the model input is scaled to $\mathbf{x} = 5\mathbf{u}$, giving inputs in $[-5, 5]^8$. For MNIST, the same mean-centered inputs as in the main task are multiplied by 5. These diagnostics are used to test whether the perturbation estimators respond differently when the incoming activation norms are increased without otherwise changing the task structure.

Table 3: Learning problems, preprocessing, and network architectures used in the experiments.

Task	**Objective / score**	**Input / output**	**Samples used**	**Input preprocessing**	**Architecture**
Sinus regression	MSE / R^2	$8 \to 1$	512 train, 512 test	Uniform inputs in $[-1, 1]^8$. Training labels include Gaussian noise with standard deviation 0.1; test labels are noiseless.	8-8-8-1, sigmoid hidden activations, linear output.
California housing	MSE / R^2	$8 \to 1$	16,512 train, 4,128 test	80/20 split. Inputs and targets standardized using training-split statistics.	8-128-64-1, sigmoid hidden activations, linear output.
MNIST	MSE / accuracy	$784 \to 10$	20,000 train, 5,000 test; 4,000 for train evaluation	Pixels divided by 255, scalar training mean subtracted, images flattened, labels one-hot encoded.	784-256-128-10, ReLU hidden activations, linear output.
CIFAR-10	MSE / accuracy	$3072 \to 10$	20,000 train, 5,000 test; 4,000 for train evaluation	Pixels divided by 255, scalar training mean subtracted, images flattened, labels one-hot encoded.	3072-512-256-10, ReLU hidden activations, linear output.

3.4 Algorithms

All perturbation-based methods were trained using mini-batches. For a mini-batch $\mathcal{B}$ of size m, the implementation first performs one noisy forward pass and computes a per-sample loss ℓ_i. The scalar learning signal is the centred reward

$$\begin{aligned} r_i &= -\ell_i - \frac{1}{m}\sum_{j=1}^{m}(-\ell_j) \\ &= \frac{1}{m}\sum_{j=1}^{m}\ell_j - \ell_i, \end{aligned} \tag{12}$$

so the baseline is the mean reward of the same noisy mini-batch. No additional noiseless baseline forward pass is used. The updates below match the implementation in `learning_rules_MLP.py`; ϵ_{num} denotes a small numerical stabilizer.

This batch-mean baseline is a practical implementation choice. In the literature considered here, baseline subtraction is typically implemented using an additional forward pass. Standard WP/NP methods often subtract the loss from a noiseless forward pass (Werfel et al., 2005; Hiratani et al., 2022; Dalm et al., 2024; Züge et al., 2023), while noisy-baseline variants subtract the loss from a second independently perturbed forward pass (Cho et al., 2011; Hara et al., 2017). Both approaches require evaluating an additional baseline trajectory. In the implementation used here, the baseline is instead computed from the same noisy mini-batch. Since each individual update uses the batch mean as its centering term, perturbations with lower-than-average loss are reinforced, while perturbations with higher-than-average loss are reversed. This avoids an additional baseline forward pass and keeps all methods in the same score-function-style family of perturbation estimators.

Algorithm 1 Weight perturbation (WP)

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^{L}$, epochs E, mini-batch size B, learning rate η, noise scale σ

for $e = 1, \ldots, E$ **do**
  for mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
    $m \leftarrow |\mathcal{B}|$
    for sample $i = 1, \ldots, m$ **do**
      $a_{0,i} \leftarrow x_i$
      for layer $k = 1, \ldots, L$ **do**
        Draw $E^W_{k,i} \sim \mathcal{N}(0, \sigma^2 I)$, $E^b_{k,i} \sim \mathcal{N}(0, \sigma^2 I)$
        $z_{k,i} \leftarrow (W_k + E^W_{k,i})a_{k-1,i} + b_k + E^b_{k,i}$
        $a_{k,i} \leftarrow \phi_k(z_{k,i})$
      end for
      $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$
    end for
    $r_i \leftarrow \frac{1}{m}\sum_{j=1}^{m}\ell_j - \ell_i$ for all i
    for layer $k = 1, \ldots, L$ **do**
      $\Delta W_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} r_i E^W_{k,i}/(\sigma^2 + \epsilon_{\text{num}})$
      $\Delta b_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} r_i E^b_{k,i}/(\sigma^2 + \epsilon_{\text{num}})$
      $W_k \leftarrow W_k + \eta\Delta W_k, \quad b_k \leftarrow b_k + \eta\Delta b_k$
    end for
  end for
end for

Algorithm 2 Vanilla node perturbation with fixed pre-activation noise

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^{L}$, epochs E, mini-batch size B, learning rate η, noise scale σ

for $e = 1, \dots, E$ **do**
  for mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
    $m \leftarrow |\mathcal{B}|$
    for sample $i = 1, \dots, m$ **do**
      $a_{0,i} \leftarrow x_i$
      for layer $k = 1, \dots, L$ **do**
        Draw $\xi_{k,i} \sim \mathcal{N}(0, I)$, set $s_{k,i} \leftarrow \sigma$
        $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i}\xi_{k,i}$
        $a_{k,i} \leftarrow \phi_k(z_{k,i})$
      end for
      $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$
    end for
    $r_i \leftarrow \frac{1}{m}\sum_{j=1}^{m} \ell_j - \ell_i$ for all i
    for layer $k = 1, \dots, L$ **do**
      $q_{k,i} \leftarrow r_i \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$ for all i
      $\Delta W_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} q_{k,i} a_{k-1,i}^{\top}$
      $\Delta b_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} q_{k,i}$
      $W_k \leftarrow W_k + \eta \Delta W_k, \quad b_k \leftarrow b_k + \eta \Delta b_k$
    end for
  end for
end for

Algorithm 3 Fan-in scaled node perturbation

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^{L}$, layer input dimensions d_{k-1}, epochs E, mini-batch size B, learning rate η, noise scale σ

for $e = 1, \dots, E$ **do**
  for mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
    $m \leftarrow |\mathcal{B}|$
    for sample $i = 1, \dots, m$ **do**
      $a_{0,i} \leftarrow x_i$
      for layer $k = 1, \dots, L$ **do**
        Draw $\xi_{k,i} \sim \mathcal{N}(0, I)$
        $s_{k,i} \leftarrow \sigma\sqrt{d_{k-1} + 1}$ ▷ includes the bias input
        $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i}\xi_{k,i}$
        $a_{k,i} \leftarrow \phi_k(z_{k,i})$
      end for
      $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$
    end for
    $r_i \leftarrow \frac{1}{m}\sum_{j=1}^{m} \ell_j - \ell_i$ for all i
    for layer $k = 1, \dots, L$ **do**
      $q_{k,i} \leftarrow r_i \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$ for all i
      $\Delta W_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} q_{k,i} a_{k-1,i}^{\top}$
      $\Delta b_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} q_{k,i}$
      $W_k \leftarrow W_k + \eta \Delta W_k, \quad b_k \leftarrow b_k + \eta \Delta b_k$
    end for
  end for
end for

Algorithm 4 Input-scaled node perturbation (IS-NP)

Require: Training set $\mathcal{D}$, model parameters $\{W_k, b_k\}_{k=1}^{L}$, epochs E, mini-batch size B, learning rate η, noise scale σ

for $e = 1, \dots, E$ **do**
  for mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
    $m \leftarrow |\mathcal{B}|$
    for sample $i = 1, \dots, m$ **do**
      $a_{0,i} \leftarrow x_i$
      for layer $k = 1, \dots, L$ **do**
        Draw $\xi_{k,i} \sim \mathcal{N}(0, I)$
        $s_{k,i} \leftarrow \sigma\sqrt{1 + \|a_{k-1,i}\|_2^2}$ ▷ WP-induced pre-activation scale
        $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i}\xi_{k,i}$
        $a_{k,i} \leftarrow \phi_k(z_{k,i})$
      end for
      $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$
    end for
    $r_i \leftarrow \frac{1}{m}\sum_{j=1}^{m} \ell_j - \ell_i$ for all i
    for layer $k = 1, \dots, L$ **do**
      $q_{k,i} \leftarrow r_i \xi_{k,i}/(s_{k,i} + \epsilon_{\text{num}})$ for all i
      $\Delta W_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} q_{k,i} a_{k-1,i}^{\top}$
      $\Delta b_k \leftarrow \frac{1}{m}\sum_{i=1}^{m} q_{k,i}$
      $W_k \leftarrow W_k + \eta \Delta W_k, \quad b_k \leftarrow b_k + \eta \Delta b_k$
    end for
  end for
end for

3.5 Hyperparameter selection and experimental protocol

Hyperparameter tuning is especially important for perturbation-based learning methods because their performance depends strongly on both the learning rate and the perturbation scale. The learning rate controls the size of the parameter update, while the perturbation scale controls the size of the random changes used to estimate the gradient. If the perturbation scale is too small, the resulting loss differences can become dominated by numerical noise or minibatch variability; if it is too large, the perturbation no longer provides a useful local estimate of the gradient. Similarly, a learning rate that is too small can make learning unnecessarily slow, while a learning rate that is too large can make the already noisy updates unstable. A fair comparison therefore requires tuning each method carefully rather than relying on a single shared setting.

For each learning problem, the dataset split, preprocessing, network architecture, batch size, and evaluation procedure are held fixed across learning rules. This ensures that differences in performance primarily reflect the learning rule rather than differences in model capacity or data processing. The hyperparameters tuned separately for each method are the learning rate and, for perturbation-based methods, the perturbation scale σ. Backpropagation only requires tuning the learning rate, while WP, vanilla NP, fan-in-scaled NP, and IS-NP each require tuning both the learning rate and perturbation scale. Each method is therefore given its own best hyperparameter setting for each task, since the goal is to compare well-tuned versions of the learning rules rather than force them to use identical hyperparameters.

The tuning procedure was carried out in two stages. First, the perturbation scale was tuned using the frozen-estimator diagnostics described above. For each task, a backpropagation model was trained and frozen at several checkpoints, and each perturbation method was evaluated over a range of candidate σ values. The main criteria in this stage were high cosine similarity and low estimator variance. The search started from broad ranges and was then narrowed until three nearby σ values remained for each method. These values formed the perturbation-scale axis of the subsequent training sweep.

Second, each method was trained over a local grid of learning rates and perturbation scales. For

the perturbation methods this gave a 3×3 grid, formed as the Cartesian product of three candidate learning rates and three candidate σ values. Backpropagation was tuned over learning rate only. The sweep runs were shorter than the final training runs, so that several nearby settings could be compared efficiently. The grid was then refined by inspecting the resulting learning curves, final losses, convergence behaviour, and signs of instability. In several cases the final learning rate was chosen as a conservative scaled version of the best grid region, especially for the noisier image tasks, while the perturbation scale was kept close to the value suggested by the frozen-estimator search. Table 4 shows the local grids used during tuning and the final settings used for the thesis results.

Table 4: Local hyperparameter grids used during tuning. For perturbation methods, the grid is the Cartesian product of the listed learning rates and perturbation scales. The selected setting is the final configuration used for the thesis results; some learning rates are conservative scaled refinements of the displayed grid region.

Task	**Method**	**Learning-rate grid**	**σ grid**	**Selected setting**
Sinus	BP	$\{0.500, 0.100, 0.150\}$	–	$\eta = 0.100$
Sinus	IS-NP	$\{0.200, 0.225, 0.250\}$	$\{0.150, 0.175, 0.200\}$	$\eta = 0.150,\ \sigma = 0.150$
Sinus	Fan-in NP	$\{0.150, 0.175, 0.200\}$	$\{0.100, 0.125, 0.150\}$	$\eta = 0.1125,\ \sigma = 0.100$
Sinus	Vanilla NP	$\{0.100, 0.125, 0.150\}$	$\{0.300, 0.375, 0.450\}$	$\eta = 0.0825,\ \sigma = 0.300$
Sinus	WP	$\{0.050, 0.075, 0.100\}$	$\{0.200, 0.225, 0.250\}$	$\eta = 0.0675,\ \sigma = 0.225$
California housing	BP	$\{0.075, 0.100, 0.150\}$	–	$\eta = 0.100$
California housing	IS-NP	$\{0.020, 0.030, 0.040\}$	$\{0.080, 0.100, 0.120\}$	$\eta = 0.0270,\ \sigma = 0.100$
California housing	Fan-in NP	$\{0.015, 0.020, 0.025\}$	$\{0.045, 0.0525, 0.060\}$	$\eta = 0.0175,\ \sigma = 0.0525$
California housing	Vanilla NP	$\{0.020, 0.025, 0.030\}$	$\{0.400, 0.450, 0.500\}$	$\eta = 0.0175,\ \sigma = 0.450$
California housing	WP	$\{0.010, 0.0125, 0.015\}$	$\{0.080, 0.100, 0.120\}$	$\eta = 0.0135,\ \sigma = 0.100$
MNIST	BP	$\{0.800, 0.900, 0.950, 1.00, 1.10, 1.20, 1.30, 2.00, 4.00\}$	–	$\eta = 0.500$
MNIST	IS-NP	$\{0.110, 0.120, 0.130\}$	$\{0.030, 0.0325, 0.035\}$	$\eta = 0.0510,\ \sigma = 0.0325$
MNIST	Fan-in NP	$\{0.030, 0.0325, 0.035\}$	$\{0.0080, 0.00875, 0.0095\}$	$\eta = 0.0122,\ \sigma = 0.00875$
MNIST	Vanilla NP	$\{0.02125, 0.0225, 0.02375\}$	$\{0.110, 0.120, 0.130\}$	$\eta = 0.00563,\ \sigma = 0.120$
MNIST	WP	$\{0.01375, 0.0150, 0.01625\}$	$\{0.025, 0.0275, 0.030\}$	$\eta = 0.00375,\ \sigma = 0.0275$
CIFAR-10	BP	$\{0.200\}$	–	$\eta = 0.100$
CIFAR-10	IS-NP	$\{0.0283, 0.0300, 0.0317\}$	$\{0.016, 0.0175, 0.01875\}$	$\eta = 0.0120,\ \sigma = 0.0175$
CIFAR-10	Fan-in NP	$\{0.0250, 0.0267, 0.0283\}$	$\{0.0050, 0.0055, 0.0060\}$	$\eta = 0.00667,\ \sigma = 0.0055$
CIFAR-10	Vanilla NP	$\{0.0142, 0.0150, 0.0158\}$	$\{0.165, 0.190, 0.215\}$	$\eta = 0.00400,\ \sigma = 0.190$
CIFAR-10	WP	$\{0.00550, 0.00600, 0.00650\}$	$\{0.011, 0.0125, 0.015\}$	$\eta = 0.00160,\ \sigma = 0.0125$

Table 5: Training and estimator-diagnostic settings for each learning problem.

Task	Train batch	Eval batch	Run epochs	Diagnostic checkpoints	Perturbations
Sinus regression	64	512	500	Epochs 1, 120, and 240 of a 240-epoch BP diagnostic run.	240
California housing	256	4096	200	Epochs 1, 25, and 50 of a 50-epoch BP diagnostic run.	200
MNIST	128	1024	400	Epochs 1, 10, and 20 of a 20-epoch BP diagnostic run.	100
CIFAR-10	128	1024	80	Epochs 1, 25, and 50 of a 50-epoch BP diagnostic run.	10

Finally, the selected configurations were run across multiple random seeds to check that the conclusions were not driven by a single favourable or unfavourable initialization.

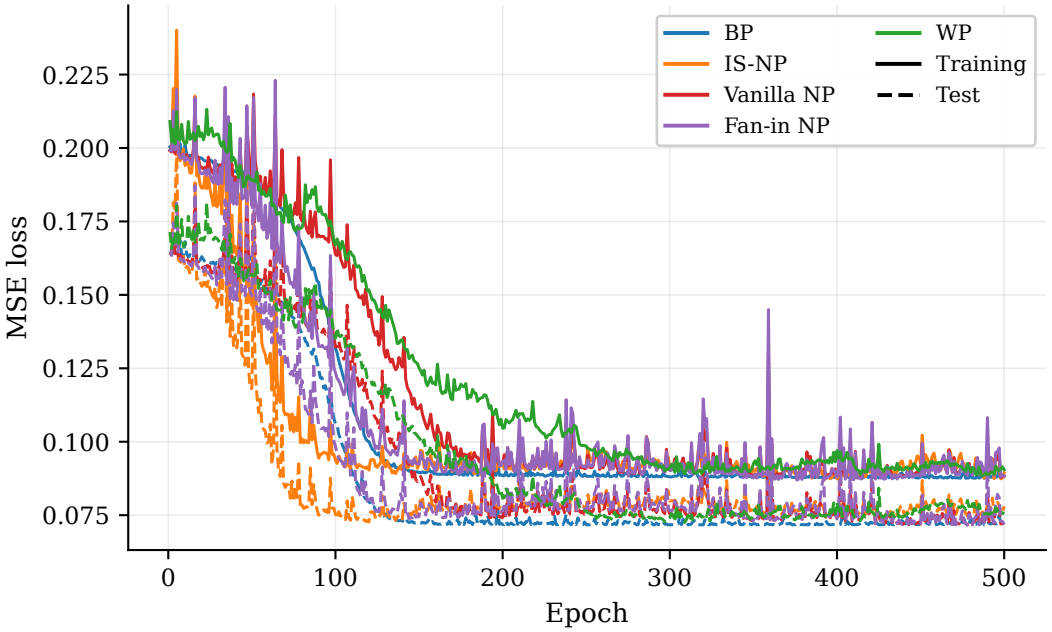
4 Results

4.1 Sinus Regression

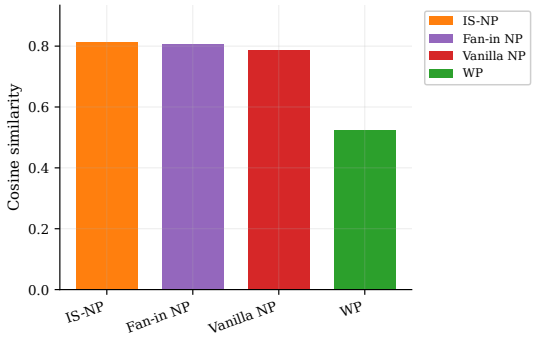
All methods reduce the training and test loss on the sinus regression task. The final losses are very similar across methods, with best test losses between 0.0716 and 0.0727. Backpropagation reaches a best test loss of 0.0717, while IS-NP, vanilla NP, fan-in NP, and WP reach 0.0727, 0.0717, 0.0716, and 0.0725, respectively. The corresponding test R^2 values are also close, ranging from 0.553 to 0.559. In terms of convergence speed, IS-NP reaches its convergence criterion earliest among the perturbation methods, at epoch 73, while WP is slowest, converging at epoch 200.

The estimator diagnostics show larger differences than the loss curves. The node perturbation methods have substantially higher cosine similarity than WP. IS-NP, fan-in NP, and vanilla NP obtain cosine similarities of 0.812, 0.808, and 0.788, respectively, while WP obtains 0.524. The same pattern appears in estimator variance. IS-NP, vanilla NP, and fan-in NP have variances of 0.00649, 0.00717, and 0.00726, respectively, whereas WP has a substantially larger variance of 0.0555.

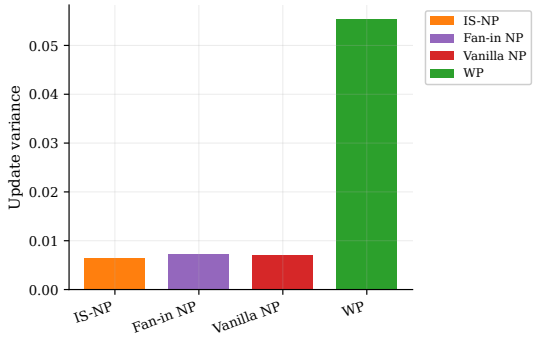
For the sinus regression task, the differences in estimator quality are therefore much larger than the differences in final training and test performance. All methods are able to learn the task to a similar final loss, but the perturbation estimators differ clearly in directional alignment and variance. The node perturbation methods show better estimator diagnostics than WP, while IS-NP has the highest cosine similarity and the lowest variance among the perturbation-based methods.



(a) Training and test loss.



(b) Cosine similarity.



(c) Estimator variance.

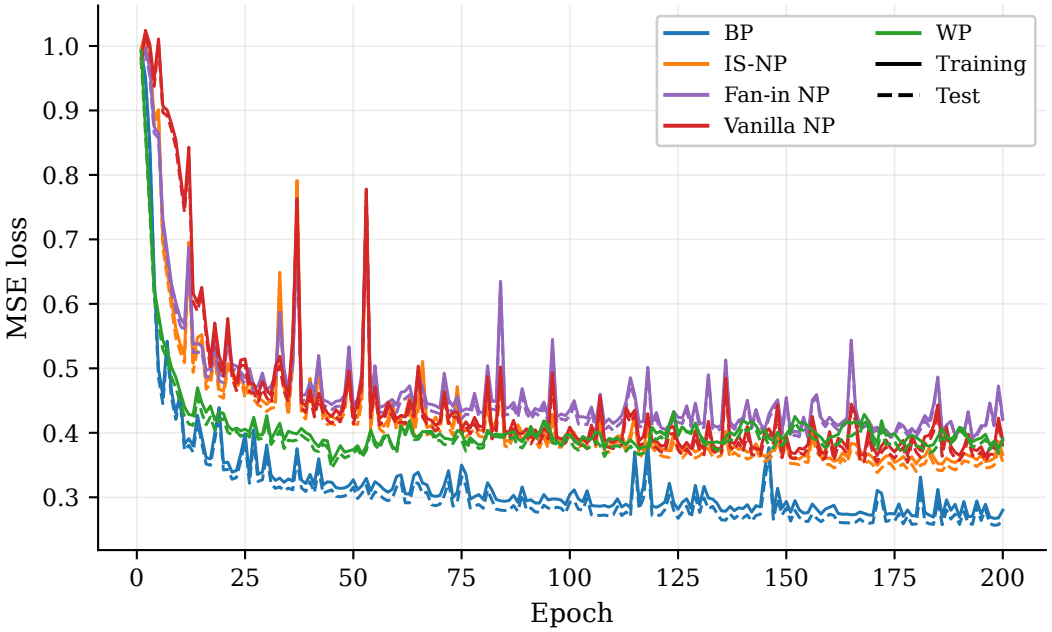
Figure 1: Results for the sinus regression task.

4.2 California Housing Regression

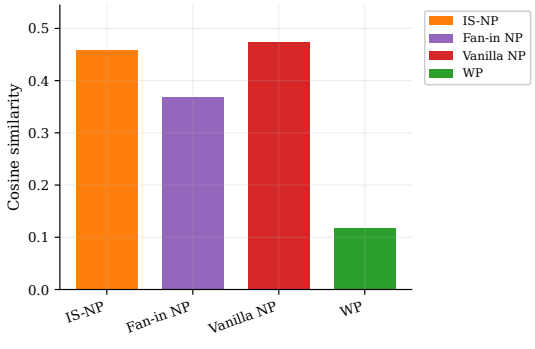
Backpropagation reaches the lowest training and test loss on the California housing task. It obtains a best test loss of 0.256, corresponding to a test R^2 of 0.741. Among the perturbation methods, IS-NP obtains the lowest test loss, with 0.338 and a test R^2 of 0.659. WP follows with a test loss of 0.345, vanilla NP reaches 0.354, and fan-in NP reaches 0.390. WP reaches its convergence criterion earliest among all methods, at epoch 18, while IS-NP converges later at epoch 54.

The estimator diagnostics show clearer separation between the methods. Vanilla NP has the highest cosine similarity, with 0.473, followed closely by IS-NP with 0.459. Fan-in NP obtains 0.368, while WP is substantially lower at 0.118. The variance ranking is different: IS-NP has the lowest estimator variance at 0.0162, followed by vanilla NP at 0.0200, fan-in NP at 0.0847, and WP at 0.792. Thus, WP has the weakest estimator diagnostics despite converging quickly in terms of training loss.

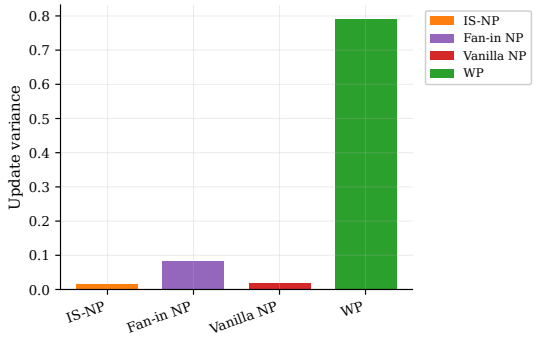
For the California housing task, the relationship between estimator diagnostics and final performance is less direct than on the harder classification tasks. IS-NP has the lowest variance and the best final performance among the perturbation methods, while vanilla NP has slightly higher cosine similarity but worse final loss. WP has low cosine similarity and high variance, but still reaches a competitive final loss and the fastest convergence criterion. Overall, the perturbation methods all learn meaningful predictors, but the ranking between them is less clean than on the more complex tasks.



(a) Training and test loss.



(b) Cosine similarity.



(c) Estimator variance.

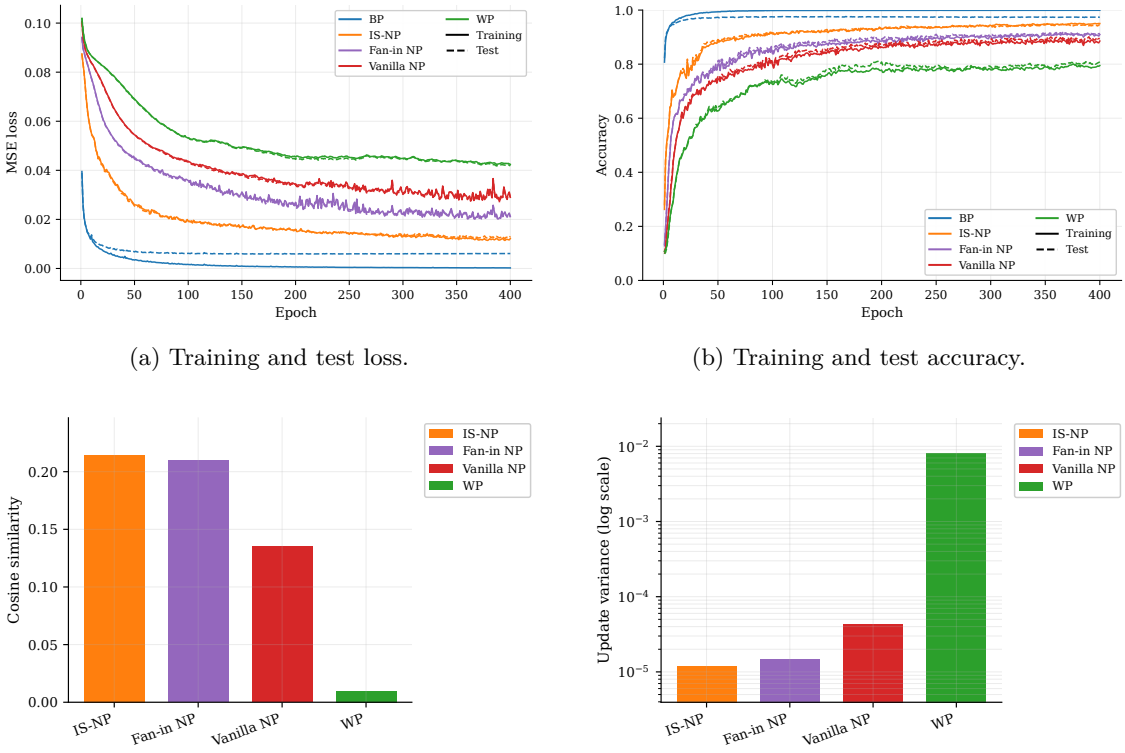
Figure 2: Results for the California housing regression task.

4.3 MNIST Classification

Backpropagation achieves the best performance on the MNIST classification task, reaching a best test loss of 0.00588 and a best test accuracy of 0.976. Among the perturbation methods, IS-NP performs best, with a best test loss of 0.0122 and a best test accuracy of 0.949. Fan-in NP follows with a test loss of 0.0199 and accuracy of 0.917, while vanilla NP reaches 0.0270 and 0.901. WP performs worst among the perturbation methods, with a test loss of 0.0418 and test accuracy of 0.810. IS-NP also converges earlier than the other node perturbation methods, reaching the convergence criterion at epoch 94, compared with 169 for fan-in NP and 189 for vanilla NP.

The estimator diagnostics show a clear separation between WP and the node perturbation methods. IS-NP and fan-in NP have the highest cosine similarities, with 0.215 and 0.210, respectively. Vanilla NP is lower at 0.135, while WP is close to zero at 0.00973. The variance results show the same overall pattern. IS-NP has the lowest estimator variance at $1.18 \cdot 10^{-5}$, followed by fan-in NP at $1.48 \cdot 10^{-5}$, vanilla NP at $4.33 \cdot 10^{-5}$, and WP at 0.00793.

For MNIST, the relationship between estimator diagnostics and learning performance is clear. The methods with higher cosine similarity and lower estimator variance also obtain better training and test performance. IS-NP has the strongest diagnostics among the perturbation methods and also reaches the best perturbation-based loss and accuracy. WP has the weakest diagnostics and the weakest final performance.



(a) Training and test loss.

(b) Training and test accuracy.

(c) Cosine similarity.

(d) Estimator variance.

Figure 3: Results for the MNIST classification task.

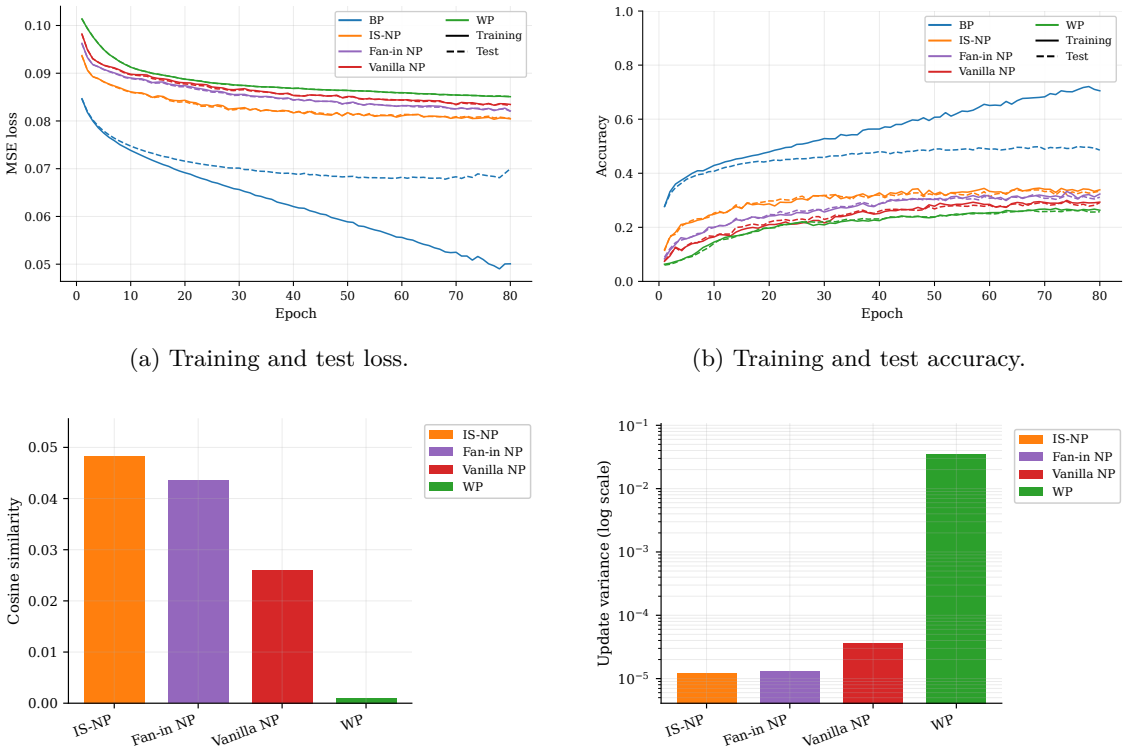
4.4 CIFAR-10 Classification

Backpropagation achieves the best performance on the CIFAR-10 classification task, reaching a best test loss of 0.0678 and a best test accuracy of 0.499. The perturbation methods reduce the loss, but plateau at substantially worse values. Among the perturbation methods, IS-NP performs best, with a best test loss of 0.0806 and a best test accuracy of 0.339. Fan-in NP follows with a test loss of 0.0820 and accuracy of 0.321, while vanilla NP reaches 0.0832 and 0.296. WP performs

worst, with a test loss of 0.0851 and test accuracy of 0.264.

The estimator diagnostics show the same ordering as the learning curves. IS-NP has the highest cosine similarity among the perturbation methods, with 0.0483, followed by fan-in NP at 0.0436, vanilla NP at 0.0260, and WP at $9.92 \cdot 10^{-4}$. The variance results also separate WP clearly from the node perturbation methods. IS-NP has the lowest variance at $1.22 \cdot 10^{-5}$, followed by fan-in NP at $1.31 \cdot 10^{-5}$, vanilla NP at $3.65 \cdot 10^{-5}$, and WP at 0.0355.

For CIFAR-10, the relationship between estimator diagnostics and learning performance is clear. The methods with higher cosine similarity and lower estimator variance also reach better training and test performance. Compared with the simpler tasks, the differences between perturbation methods are more visible, and the gap between all perturbation methods and backpropagation is substantially larger.



(a) Training and test loss.

(b) Training and test accuracy.

(c) Cosine similarity.

(d) Estimator variance.

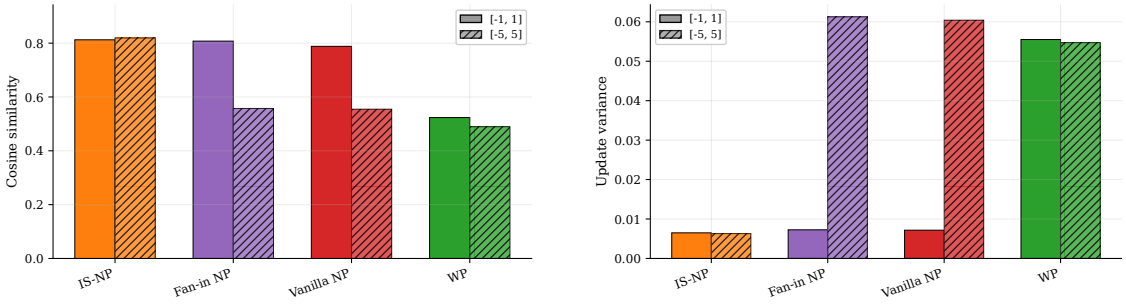
Figure 4: Results for the CIFAR-10 classification task.

4.5 Scaled Input Sensitivity

Scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$ changes the estimator diagnostics differently across methods. IS-NP and WP are only weakly affected by the input scaling, while vanilla NP and fan-in NP degrade substantially. For vanilla NP and fan-in NP, cosine similarity decreases clearly under the scaled input setting, whereas IS-NP remains close to its original level and WP changes comparatively little. The same pattern appears in estimator variance: vanilla NP and fan-in NP show a large increase in variance, while IS-NP and WP remain comparatively stable.

The scaled-input experiment therefore separates the methods more clearly than the original sinus regression task. In the default $[-1, 1]$ setting, all node perturbation methods have relatively high cosine similarity and low estimator variance. Under input scaling, the diagnostics for vanilla NP and fan-in NP worsen, while IS-NP remains the strongest node perturbation method. This result is consistent across both diagnostic metrics shown in Figure 5: the methods that account for the actual effect of input scale, WP and IS-NP, are less sensitive to the change than methods using

fixed or layer-width-based node perturbation scales.



(a) Cosine similarity.

(b) Estimator variance.

Figure 5: Effect of scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$.

Table 6: Summary of training performance, estimator alignment, estimator variance, and convergence speed.

task	method	best_train_loss	best_test_loss	best_train_score	best_test_score	score_metric	convergence_epoch	cosine	variance
Sinus	BP	0.0876	0.0717	0.555	0.559	R2	118	1.00	0.00
Sinus	IS-NP	0.0873	0.0727	0.556	0.553	R2	73	0.812	0.00649
Sinus	Vanilla NP	0.0876	0.0717	0.554	0.558	R2	159	0.788	0.00717
Sinus	Fan-in NP	0.0870	0.0716	0.558	0.559	R2	119	0.808	0.00726
Sinus	WP	0.0883	0.0725	0.551	0.554	R2	200	0.524	0.0555
California Housing	BP	0.267	0.256	0.733	0.741	R2	22	1.00	0.00
California Housing	IS-NP	0.350	0.338	0.650	0.659	R2	54	0.459	0.0162
California Housing	Fan-in NP	0.392	0.390	0.608	0.607	R2	43	0.368	0.0847
California Housing	Vanilla NP	0.363	0.354	0.637	0.643	R2	45	0.473	0.0200
California Housing	WP	0.356	0.345	0.644	0.652	R2	18	0.118	0.792
MNIST	BP	2.25e-4	0.00588	0.999	0.976	Accuracy	21	1.00	0.00
MNIST	IS-NP	0.0114	0.0122	0.951	0.949	Accuracy	94	0.215	1.18e-5
MNIST	Fan-in NP	0.0202	0.0199	0.913	0.917	Accuracy	169	0.210	1.48e-5
MNIST	Vanilla NP	0.0276	0.0270	0.890	0.901	Accuracy	189	0.135	4.33e-5
MNIST	WP	0.0424	0.0418	0.801	0.810	Accuracy	164	0.00973	0.00793
CIFAR-10	BP	0.0490	0.0678	0.721	0.499	Accuracy	34	1.00	0.00
CIFAR-10	IS-NP	0.0804	0.0806	0.345	0.339	Accuracy	40	0.0483	1.22e-5
CIFAR-10	Fan-in NP	0.0821	0.0820	0.333	0.321	Accuracy	53	0.0436	1.31e-5
CIFAR-10	Vanilla NP	0.0833	0.0832	0.300	0.296	Accuracy	52	0.0260	3.65e-5
CIFAR-10	WP	0.0851	0.0851	0.270	0.264	Accuracy	43	9.92e-4	0.0355

5 Discussion

5.1 Overview of the main findings

The discussion interprets the results in relation to the objectives of the thesis. The first objective, to establish the current state of perturbation-based learning, was addressed in Chapter 2 through the review and comparison of existing perturbation methods. The discussion therefore focuses mainly on the two remaining questions: what the empirical results show about the practical capabilities and limitations of perturbation-based learning, and whether the proposed IS-NP method improves on existing perturbation-based methods.

The results show that perturbation-based methods can learn on all tasks considered, but that their performance relative to backpropagation decreases as task complexity increases. On the simplest regression task, all methods reach similar final performance. On the more difficult tasks, the gap to backpropagation becomes larger, and the differences between perturbation methods become clearer. This supports the view that perturbation methods are viable in simple and moderately difficult settings, but that their noisy gradient estimates become a major limitation as the learning problem becomes more complex.

The results also provide empirical support for the theoretical motivation behind IS-NP. The variance reduction theorem showed that IS-NP is matched to WP in expectation while having no larger coordinate-wise estimator variance. Empirically, IS-NP consistently has much lower variance than WP and generally performs better than the other perturbation-based methods, especially on the classification tasks. The scaled-input experiment further supports the input-scaling argument, as IS-NP is less sensitive than vanilla NP and fan-in NP when activation scales change.

5.2 Practical capability and limits of perturbation-based learning

The empirical results show that perturbation-based learning methods are capable of solving simple and moderately difficult supervised learning problems, but that their performance degrades relative to backpropagation as task complexity increases. On the sinus regression task, all perturbation methods are able to reduce the loss effectively and converge to solutions close to those obtained by backpropagation. This shows that, in sufficiently simple settings, scalar-feedback perturbation methods can provide useful learning signals despite relying on noisy gradient estimates. The California housing task gives a similar overall picture: the perturbation methods are able to learn meaningful predictors on a real tabular dataset, although the differences between methods are less consistent and backpropagation remains the more efficient reference method.

The gap between perturbation-based learning and backpropagation becomes clearer on the image classification tasks. On MNIST, the perturbation methods still learn useful representations, and IS-NP in particular reaches relatively strong performance compared with the other perturbation methods. However, learning is slower than with backpropagation, and the final performance remains worse. This suggests that MNIST is around the level of task complexity where the strongest perturbation method tested here is still capable of producing good empirical results, but where the advantage of exact gradient information already becomes clear.

CIFAR-10 exposes the current limitations of the perturbation methods more strongly. Although the perturbation-based methods are able to learn some structure from the data, their training and test performance remain far below backpropagation under the MLP-based experimental setup used in this thesis. This should not be interpreted as a failure of the experiment. The task progression was deliberately chosen to include cases where the methods begin to break down, because identifying these limits is part of assessing their practical viability. In this setting, CIFAR-10 appears to be beyond what the tested perturbation methods can solve well without additional architectural or optimization improvements.

The estimator diagnostics help explain this trend. As the tasks become more difficult, the perturbation-based estimators generally show weaker directional alignment with the backpropagation gradient

and higher estimator variance. This means that the updates become less reliably directed toward reducing the loss, while also fluctuating more strongly between perturbation samples. Backpropagation avoids this problem by computing the exact gradient directly, which explains why the performance gap grows on harder tasks. Thus, the main empirical limitation of perturbation-based learning is not that it cannot learn at all, but that its noisy gradient estimates become increasingly inefficient as the learning problem becomes more complex.

Overall, the results suggest that perturbation methods are viable for relatively simple tasks and can remain useful on moderately complex problems, but they are not competitive with backpropagation across the full range of tasks considered here. This is consistent with the motivation of the thesis: perturbation methods are attractive because they are simple, local, and biologically plausible, not because they are expected to outperform exact gradient descent in standard artificial neural networks. Their practical value therefore depends on whether their simplicity and biological plausibility can justify the loss in efficiency, and whether theoretically motivated improvements such as IS-NP can extend the range of tasks they are able to solve.

5.3 Empirical support for IS-NP

The empirical results provide strong support for the proposed input-scaled node perturbation method. Across the full set of experiments, IS-NP is generally the best-performing perturbation-based method, particularly on the more complex tasks where differences in estimator quality become most important. This supports the second objective of the thesis, namely to further develop perturbation-based learning through a theoretically motivated modification and assess whether this modification improves practical performance.

The clearest support comes from the comparison with weight perturbation. The theory developed in Chapter 2 showed that IS-NP matches WP in expectation while having no larger coordinate-wise estimator variance. This prediction is strongly reflected in the empirical results. In practice, IS-NP consistently exhibits much lower variance than WP, and this is typically accompanied by better directional alignment with the backpropagation gradient, faster learning, and better final performance. Thus, the theoretical variance reduction is not only a formal result, but also appears to translate into a clear practical advantage.

The comparison with the other node perturbation variants is also encouraging. Overall, the empirical results suggest the ranking

$$\text{IS-NP} > \text{fan-in NP} > \text{vanilla NP} > \text{WP},$$

especially on MNIST and CIFAR-10. On these more demanding tasks, IS-NP converges faster, reaches a better final solution, attains higher cosine similarity, and exhibits lower variance than the other perturbation methods. Fan-in-scaled NP is generally the second-best method, which is consistent with the idea that incorporating some information about input scale is better than using a completely fixed perturbation scale. Vanilla NP is weaker, and WP is generally the weakest method among the perturbation-based baselines.

At the same time, the results also show that these differences are not equally visible on all tasks. On the simpler problems, especially sinus regression and to some extent California housing, the perturbation methods often perform quite similarly in terms of final training and test performance. In such settings, the tasks appear simple enough that all methods are able to find good solutions, even if their estimator quality differs. This likely explains why the ranking is less clear there, and why WP can even appear competitive in some cases, for example in terms of convergence speed on California housing. These exceptions do not contradict the theoretical and empirical story, but rather suggest that when the task is easy enough, differences in estimator quality may matter less for overall optimization performance.

The strongest evidence for the mechanism behind IS-NP comes from the scaled-input experiments. These results show that vanilla NP and fan-in-scaled NP degrade much more severely when the norm of the incoming activations changes substantially, while IS-NP and WP remain comparatively robust. This is exactly what would be expected from the theoretical construction. Vanilla NP

implicitly assumes that a fixed perturbation scale is appropriate across inputs and layers, while fan-in NP adjusts only for layer width. IS-NP instead uses the actual incoming activation norm, and therefore continues to apply perturbations on the correct scale even when those assumptions break down. The empirical results therefore support not only the claim that IS-NP performs better, but also the proposed explanation for why it performs better.

Taken together, the experiments suggest that IS-NP is a meaningful improvement over the most relevant existing perturbation-based baselines considered in this thesis. The advantage is small when the task is very simple, but becomes increasingly clear as the learning problem becomes more demanding or when activation scales vary in ways that challenge the assumptions of simpler NP variants. This does not mean that IS-NP solves the broader limitations of perturbation-based learning, but it does show that the theoretically motivated modification proposed in this thesis pushes this class of methods in a more capable direction.

5.4 Why input scaling matters

The results suggest that input scaling matters because node perturbation is ultimately used to estimate weight-level updates. In vanilla NP, the same pre-activation perturbation scale is used regardless of the magnitude of the incoming activation vector. This is simple, but it ignores how strongly a perturbation at the weight level would affect the corresponding pre-activation. If the incoming activations are small, fixed-scale NP can create a large pre-activation change while assigning only a small weight update, since the update is proportional to the presynaptic activity. If the incoming activations are large, the opposite can happen: the node perturbation may be too small relative to the effect that weight perturbation would naturally induce.

Fan-in-scaled NP partially addresses this problem by scaling the perturbation according to the number of incoming units. This is a reasonable expectation-level correction: under simplifying assumptions, the norm of the incoming activation vector grows with the square root of the layer width. The empirical results also support this interpretation, since fan-in-scaled NP is usually stronger than vanilla NP. However, fan-in scaling still does not observe the actual activation norm. It assumes that layer width is a sufficient proxy for the scale of the incoming vector, which may be inaccurate across samples, layers, datasets, and preprocessing choices.

IS-NP avoids this approximation by scaling the node perturbation using the actual norm of the incoming activations. This makes the perturbation scale sample-dependent and layer-dependent, rather than fixed globally or determined only by layer width. The method therefore preserves the main advantage of node perturbation, namely perturbing a lower-dimensional activity space, while matching the scale of the perturbation more closely to the effect that weight perturbation would have produced at the pre-activation level. This provides a direct explanation for why IS-NP tends to achieve better directional alignment and lower estimator variance than the simpler NP variants.

The scaled-input experiment provides the clearest evidence for this mechanism. When the input range is expanded, the assumptions behind vanilla NP and fan-in-scaled NP become less appropriate, because the activation norms no longer remain on the same scale as before. As expected, these methods show much worse estimator diagnostics and can become unstable. In contrast, WP and IS-NP are much less affected, because both methods naturally account for the scale of the incoming activations: WP through the actual effect of weight noise, and IS-NP through the induced pre-activation noise distribution. This supports the interpretation that IS-NP is not merely better tuned for one particular setting, but is more robust to changes in activation scale.

This robustness is important for the broader motivation of the thesis. In more biologically inspired architectures, it may not be reasonable to assume that all layers, neurons, or input channels operate on carefully normalized scales. A perturbation rule that depends on a fixed global noise scale may therefore be fragile. IS-NP reduces this dependence by adapting the perturbation scale to the local activity entering each neuron. This does not remove the need for hyperparameter tuning, but it makes the perturbation rule less dependent on assumptions about activation magnitudes and layer widths.

5.5 Limitations

The results should be interpreted within the scope of the experimental setup used in this thesis. All experiments use fully connected multilayer perceptrons, even for image classification tasks such as MNIST and CIFAR-10. This choice was made deliberately to keep the comparison focused on the learning rule rather than the architecture. However, it also limits the absolute performance that can be expected on image tasks. In particular, the poor perturbation-based performance on CIFAR-10 should not be interpreted as a general proof that these methods cannot learn complex image tasks, but rather as evidence that they struggle in the MLP setting considered here.

A second limitation is that the empirical comparison depends on the chosen hyperparameter search and training protocol. Perturbation methods are sensitive to both the learning rate and the perturbation scale, and better schedules or optimization tricks could improve performance. For example, reducing the learning rate might prevent instability in some methods during long training runs, while momentum or adaptive normalization could make noisy updates more usable. The hyperparameters used here were chosen to give each method a fair chance under a common experimental framework, but they do not exhaust all possible ways of training these algorithms.

The theoretical result also has a specific scope. The variance reduction theorem compares IS-NP to WP under a matched Gaussian perturbation construction, and shows that IS-NP has no larger coordinate-wise estimator variance while preserving the expected WP update. It does not prove that IS-NP must outperform vanilla NP or fan-in-scaled NP in all settings. The comparison with these methods is motivated by the input-scaling argument and supported empirically in this thesis, but remains dependent on the task, architecture, activation statistics, and hyperparameter choices.

Another limitation is that the experiments focus on standard supervised learning problems. This makes the comparison with backpropagation clear and controlled, but it does not fully capture the settings where perturbation methods may be most biologically relevant. In particular, perturbation-based learning is naturally connected to scalar reward signals, stochastic exploration, and online interaction with an environment. Reinforcement learning tasks, recurrent networks, temporal credit assignment problems, or biologically inspired hardware could therefore reveal different strengths and weaknesses than the feedforward supervised tasks studied here.

Finally, the empirical diagnostics are estimates rather than exact quantities. Cosine similarity and estimator variance are computed from a finite number of perturbation samples at selected stages of training, and therefore provide snapshots of estimator quality rather than a complete description of the learning dynamics. They are useful for explaining the observed trends, especially the difference between WP and IS-NP, but they do not capture every factor that affects optimization. Training performance ultimately depends on the interaction between estimator bias, variance, curvature, learning rate, architecture, and data distribution.

5.6 Future work

A natural direction for future work is to test perturbation-based learning methods on architectures that are better suited to the tasks considered. In this thesis, all methods were evaluated using fully connected multilayer perceptrons in order to isolate the effect of the learning rule. However, convolutional architectures would be more appropriate for image classification tasks such as MNIST and CIFAR-10. Testing IS-NP in CNNs would therefore help determine whether the performance gap to backpropagation is mainly caused by the learning rule itself, or partly by the use of architectures that are poorly matched to the data.

Another important direction is to combine perturbation methods with optimization techniques that reduce the effect of noisy updates. The experiments show that perturbation-based methods can learn, but that their updates become inefficient as task complexity increases. Methods such as momentum, adaptive learning rates, Adam-like normalization, layerwise learning-rate scaling, or variance normalization could make the stochastic estimates more usable in practice. These additions would not change the biological motivation of perturbation learning entirely, but they could substantially improve empirical performance and stability.

Future work should also investigate whether multiple perturbation samples or more structured perturbation schemes can improve estimator quality. The methods studied in this thesis primarily use single-sample perturbation estimates during training, which makes the updates noisy. Averaging several perturbation samples per update would reduce variance, although at increased computational cost. More structured approaches, such as layerwise perturbations, blockwise perturbations, antithetic sampling, or control variates, could provide better trade-offs between estimator quality and computational efficiency.

The theoretical analysis could also be extended beyond the setting considered here. The variance reduction result shows that IS-NP is a Rao–Blackwellized version of WP under a matched Gaussian perturbation construction. However, the comparison between IS-NP and other NP variants remains primarily empirical. A useful direction would therefore be to analyze more precisely when input-scaled node perturbation should outperform fixed-variance or fan-in-scaled NP, and whether similar variance-reduction arguments can be developed for other perturbation distributions, baselines, or network architectures.

Finally, perturbation methods should be tested in settings that are closer to their biological motivation. Supervised feedforward learning provides a controlled benchmark, but perturbation methods are especially natural in settings with scalar reward signals, stochastic exploration, and online interaction. Reinforcement learning, recurrent networks, temporal tasks, neuromorphic systems, or bio-hybrid architectures would therefore be important test cases. These settings may reveal whether the simplicity of perturbation-based learning can become an advantage in architectures where backpropagation is difficult or impossible to apply.

6 Conclusion

Conclusion paragraph 1: Restate motivation and thesis aim.

Conclusion paragraph 2: Objective 1: literature review clarified the perturbation-learning landscape.

Conclusion paragraph 3: Objective 2: theory contribution, IS-NP, expectation matching, lower variance.

Conclusion paragraph 4: Objective 3: empirical results, where methods work/break, IS-NP performance.

Final paragraph: What this means for biologically inspired architectures and future perturbation learning.

Appendix